

Anjuman-I-Islam's
M.H. Saboo Siddik Polytechnic
8, M.H. Saboo Siddik Polytechnic Road, Mumbai 400008



FINAL YEAR DIPLOMA IN COMPUTER ENGINEERING
(2023-2024)

PROJECT REPORT ON

PROJECT TITLE

Child Monitoring System App using Android

BY

210428 Bubere Huzaifa Shadab

210446 Mohd Saad Mohammed Saeed Shah

210450 Adnan Shaikh

UNDER THE GUIDANCE OF

MS. ZAIBUNNISA MALIK

MS. Shafaque Mam



Maharashtra State Board of Technical Education (MS-BTE)
Mumbai (Autonomous) (ISO 9001:2008) (ISO/IEC 27001:2005)



CERTIFICATE

This is to certify that the project has been duly signed
and performed by

210428- Bubere Huzaifa Shadab

210446- Mohd Saad Mohammed Saeed Shah

210450- Shaikh Adnan Mohd Rafik

of Third year diploma in computer engineering during the academic year

2023-24. They have completed the project as per the guidance

prescribed by MSBTE

HEAD OF DEPARTMENT:

INTERNAL EXAMINER:

DATE: _____

DATE: _____

SIGN OF THE GUIDE:

EXTERNAL EXAMINER:

DATE: _____

DATE: _____

PRINCIPAL: _____

DATE: _____

Acknowledgement

It is our esteemed pleasure to present the project report on **“Child Monitoring System Using Android”**

We would firstly like to thank our Principal (I/c), Head of the Department & Guide Ms. Zaibunnisa Malik for encouraging and motivating us with her guidance and total support for our work. We shall also like to thank Ms. Salima Khatib for working as our sub guide and making our path to integrity much simpler.

We also thank all the teachers who constantly motivated us and provided us their precious knowledge about the procedures carried out for making a project along with technical knowledge they have availed.

We would also like to thank our principal Mr. A.K Qureshi for providing us this Opportunity of integrating our own project and constantly supporting us throughout the process.

It would also be pleasure thanking all the staff, be it teaching or non-teaching who always understood by us and never made any problem tread our way.

Table of Content

Sr. No	Chapter	Page No.
1.	Introduction and Background 1.1. Introduction 1.2. Background 1.3. Motivation 1.4. Problem Statement 1.4.1 Problem Definition 1.4.2 Proposed Work 1.4.3 Proposed Methodology 1.5. Objective and Scope 1.5.1 Scope 1.5.2 Objective 1.6. Advantages 1.7. Disadvantages	
2.	Literature Survey 2.1. Introduction 2.2. Research Papers 2.3. References 2.4. Conclusion	

Sr. No	Chapter	Page No.
3.	System Analysis 3.1. Introduction 3.2 System Design 3.2.1. Introduction to System Planning 3.2.2. Software Design Approach 3.3. Gantt Chart 3.4. Time Line Chart 3.5. Cost Estimation 3.5.1 Cost Beneficial Analysis 3.6. Feasibility 3.7. Conclusion	
4.	System Design 4.1. Introduction 4.2 Block Diagram 4.3. System Architecture 4.4. Data Flow Diagram 4.5. Table Structure 4.6. State Transition Diagram 4.7. E-R Diagram 4.8. Conclusion	
5.	Implementation 5.1. Introduction 5.2. Algorithm 5.3. Flowchart 5.4. Coding 5.5. Conclusion	

Sr. No	Chapter	Page No.
6.	Testing and Debugging 6.1. Introduction 6.2. Testing Approach 6.3. Test Plan 6.4. Debugging Approach 6.5. Conclusion	
7.	Snapshots 7.1. Snapshots	
8.	Future Development and Conclusion 8.1. Introduction 8.2. Limitations 8.3. Future Enhancement 8.4. Reference and Bibliography	

Chapter 1: Introduction and Background

Content:

1.1.Introduction

1.2. Background

1.3.Motivation

1.4.Problem Statement

1.4.1. Problem Definition

1.4.2. Proposed Work

1.4.3. Proposed Methodology

1.5. Objectives and Scope

1.5.1. Scope

1.5.2. Objectives

1.6. Advantages

1.7. Disadvantages

1.2.Introduction

In today's World! Child Safety is a major concern for the Parents, especially those parents who are working Outside. In the World of increasing Crimes and Kidnapping of Children's the parents are worried when their Child is Outside. Child Monitoring System is designed to provide a Security and Monitoring of the Child. With its user-friendly interface and comprehensive features, the Child Monitoring System empowers parents to maintain a protective and vigilant eye over their children, ensuring their safety and well-being.

Child monitoring system app can provide parents with valuable tools to ensure their child's safety, it's important to strike a balance between monitoring and respecting a child's privacy. Open communication with children and educating them about the purpose of the app is essential for building trust and maintaining a healthy parent-child relationship.

1.2. Background

The concept of child monitoring systems has evolved over the years in response to changing societal norms, technological advancements, and concerns for child safety.

This Allow parents to create accounts and log in securely. Implement strong authentication methods to ensure data privacy and security. Child monitoring system will allow Parent's Real time tracking of their Child's location and will be able to access their Calls & messages.

In the early 2000s, the first child monitoring software applications were introduced, primarily focused on monitoring online activities and limiting access to certain websites. As mobile technology advanced, so did child monitoring systems, encompassing features like GPS tracking, Emergency Button and real-time alerts

1.3. Motivation

The Child Monitoring System was created to help Parents to track their Child when they are outside or When their Child is Outside. A Child Monitoring System serves as a crucial tool for addressing various parental concerns and ensuring the well-being of children.

Firstly, it is motivated by the paramount need for safety and security. Parents and caregivers want to safeguard their children from potential risks, whether within the home or in external environments. The worry about a child's safety and whereabouts when not in immediate sight can be a source of anxiety and stress.

Child monitoring systems alleviate this concern by allowing parents to remotely check on their child's activities, instilling a sense of reassurance. In emergency situations, child monitoring systems play a critical role. They provide rapid alerts to parents and caregivers in cases of accidents, medical issues, or unforeseen events, ensuring swift intervention and assistance.

1.4.Problem Statement

"In today's fast-paced and digitally connected world, parents and caregivers face an increasing need for effective and comprehensive child monitoring solutions. The challenges arise from the growing concerns related to child safety, privacy, and digital security.

This problem statement highlights the urgency of developing child monitoring systems that are capable of addressing these multifaceted challenges, providing parents and caregivers with the tools and insights necessary to keep children safe, secure, and thriving in today's complex and rapidly evolving world while respecting their individual autonomy and privacy. The child will be linked with the parents through some Unique ID so that except parents no other person will be able to access the child device & its information.

1.4.1. Problem Definition

With the increasing concerns regarding child safety and well-being, there is a growing demand for effective child monitoring solutions among parents and caregivers. Existing monitoring systems often lack comprehensive features, real-time tracking capabilities, and user-friendly interfaces, leading to difficulties in ensuring the safety and security of children, particularly in busy or unfamiliar environments. Addressing these limitations is crucial to provide parents with peace of mind and ensure the safety of their children in various situations.

1.4.2. Proposed Work

Our proposed work aims to develop an innovative child monitoring system, specifically designed for Android devices, named "GuardianWatch." This system will utilize a combination of GPS tracking, geofencing, and real-time communication features to provide parents with a reliable and user-friendly solution for monitoring their children's whereabouts and activities. By integrating advanced functionalities such as emergency alerts, location history tracking, and customizable notifications, GuardianWatch will

empower parents to stay informed and connected with their children at all times, enhancing child safety and parental peace of mind.

1.4.3. Proposed Methodology

Requirement Analysis: Conduct thorough research and surveys to understand the needs and concerns of parents regarding child monitoring systems. Identify key functionalities and features required for ensuring child safety and parental supervision effectively.

Technology Selection: Evaluate various technologies and APIs available for Android development, including GPS tracking libraries, messaging protocols, and user interface frameworks. Select the most suitable components based on factors such as reliability, accuracy, and compatibility with Android devices.

System Design: Develop a comprehensive system architecture for GuardianWatch, outlining the interactions between different modules such as GPS tracking, geofencing, messaging, and user interface components. Design intuitive and user-friendly interfaces for both parents and children.

Data Collection and Privacy Considerations: Establish protocols for data collection, storage, and privacy protection to ensure compliance with relevant regulations and guidelines. Implement encryption mechanisms and access controls to safeguard sensitive information and maintain user confidentiality.

App Development: Build the GuardianWatch application for Android devices, focusing on reliability, performance, and ease of use. Implement features such as real-time location tracking, geofence alerts, SOS button, and messaging capabilities to enable effective child monitoring and communication.

Testing and Validation: Conduct rigorous testing of the GuardianWatch application to assess its functionality, usability, and reliability under various scenarios. Perform unit testing, integration testing, and user acceptance testing to identify and resolve any issues or inconsistencies.

Deployment and User Training: Deploy the GuardianWatch application on the Google Play Store and other distribution platforms. Provide comprehensive user training and support to parents and caregivers to ensure they can effectively utilize the monitoring system and understand its features and capabilities.

Continuous Improvement: Gather feedback from users and stakeholders to identify areas for enhancement and refinement. Regularly update the GuardianWatch application with new features, performance improvements, and bug fixes to maintain its effectiveness and relevance in ensuring child safety and parental supervision.

1.5. Objective and Scope

❖ Objective:

- ☐ To Understand and Study the existing Software.
- ☐ To Develop an android app for Tracking & Monitoring the Child.
- ☐ To Provide an Emergency to the Child Side.
- ☐ To Provide call History & messages.

❖ Scope:

The project of Child Monitoring System serves the crucial purpose of ensuring the safety, wellbeing, and responsible upbringing of children in today's rapidly evolving digital and physical world. Child monitoring systems alleviate this concern by allowing parents to remotely check on their child's activities, instilling a sense of reassurance. In emergency situations, child monitoring systems play a critical role.

They provide rapid alerts to parents and caregivers in cases of accidents, medical issues, or unforeseen events, ensuring swift intervention and assistance. The primary Scope of a Child Monitoring System is to strike a balance between fostering independence in children while providing a safety net for concerned parents and guardians.

1.6. Advantages

- **Enhanced Safety:** Child Monitoring Systems provide a layer of safety by enabling parents to keep track of their child's whereabouts, especially in unfamiliar or potentially risky situations. GPS tracking, for example, can help locate a child in case they get lost or encounter an emergency.
- **Communication:** Many child monitoring solutions include communication features, such as instant messaging or SOS alerts, which allow parents and children to stay in touch and respond to emergencies or unexpected situations quickly.

- **Real-time Notifications:** Parents receive real-time notifications and alerts, providing immediate information about their child's activities, location, or any concerning behavior, which allows for timely intervention.
- **Peace of Mind:** Child Monitoring Systems offer parents peace of mind, knowing that they can quickly check on their child's safety and well-being, whether they are at school, with a babysitter, or out with friends. This reassurance can reduce parental anxiety.
- **Prevent Unauthorized Access:** Child Monitoring Systems can secure a child's devices, ensuring that only trusted apps and content are accessible. This helps prevent children from stumbling upon or accessing harmful or inappropriate material.

1.7. Disadvantages

The system must be given proper inputs otherwise it can produce wrong results

- **Invasion of Privacy:** Child Monitoring Systems, when not used with sensitivity, can intrude on a child's privacy, leading to feelings of mistrust and resentment. Balancing safety with respect for a child's autonomy is essential.
- **Trust Issues:** Overreliance on monitoring can lead to trust issues within the parent-child relationship. Children may feel that their parents don't trust them, which can strain the bond between them.
- **False Alarms:** Monitoring systems can generate false alarms or unnecessary notifications, causing anxiety and stress for parents and children. This can lead to overreactions and unnecessary interventions.
- **Data Security:** Storing sensitive data about a child's activities and whereabouts can pose security risks. If not adequately protected, this information could be vulnerable to hacking or unauthorized access.

1.8. Limitations

- **Ethical and Legal Issues:** Monitoring practices need to adhere to ethical and legal standards and in some cases, their use may raise legal and ethical concerns.
- **Cost:** Implementing comprehensive Child Monitoring Systems can be costly due to the need for purchasing devices, subscriptions, or services.
- **Frustration and Resistance:** Children might become frustrated or resistant to monitoring, leading to a lack of cooperation or a breakdown in communication.
- **Cultural Differences:** The acceptability and appropriateness of Child Monitoring Systems may vary among cultures and societies, potentially leading to different expectations and attitudes.
- **Parent-Child Relationship:** Overuse or misuse of monitoring can strain the parent-child relationship, affecting trust and communication.
- **Device Compatibility:** Compatibility issues may arise when trying to monitor children's activities on various devices and platforms, especially as they change or upgrade their technology.

- **Impact on Child's Self-Esteem:** Overuse of monitoring can affect a child's self-esteem and self-worth, as they may feel they are constantly being watched and judged.
- **Social Isolation:** Excessive monitoring can lead to social isolation, as children may feel isolated and unable to interact with their peers freely due to parental surveillance.
- **Availability of Device:** The device will be required on both the side in order to Monitor.

1.9.Conclusion

The Conclusion of Child Monitoring Systems can be valuable tools for parents and guardians to enhance the safety and well-being of their children in an increasingly digital and complex world. These systems offer a range of benefits, including improved safety, peace of mind, and the ability to address online and offline risks. However, it is essential to acknowledge the limitations and potential disadvantages of such systems, such as privacy concerns and their impact on trust and independence.

To harness the advantages of Child Monitoring Systems while mitigating their drawbacks, a balanced approach is crucial. This approach includes open communication with children about the reasons for monitoring, setting clear expectations, respecting privacy, and fostering trust within the parent-child relationship. By utilizing these systems responsibly and in conjunction with guidance and education, parents can create a supportive and secure environment for their children's growth and development in today's digital age.

Chapter 2: Literature Survey

Content:

- 2.1. Introduction
- 2.2. Research Papers
- 2.3. References
- 2.4. Conclusion

2.1. Introduction

This section is crucial as it provides context, sets the stage for the reader, and outlines the purpose and significance of the literature survey. It Provide background information on the subject of your literature survey. Explain why this topic is relevant and why it merits a comprehensive review of existing literature. Include any historical context or developments that have led to the need for such a survey.

Child monitoring system app can provide parents with valuable tools to ensure their child's safety, it's important to strike a balance between monitoring and respecting a child's privacy. With its user-friendly interface and comprehensive features, the Child Monitoring System empowers parents to maintain a protective and vigilant eye over their children, ensuring their safety and well-being.

2.2. Research Papers

Paper Title 1: Child Safety & Tracking Management System By using GPS, Geo-Fencing & Android Application

Author: Aditi Gupta, Vibhor Harit

Published in: 2016 IEEE

Abstract: The paper presents a child safety and tracking system that utilizes GPS, Geo-Fencing, and SMS technologies. It addresses the concern of parents regarding the security of their children and provides real-time tracking and emergency messaging features.

The proposed system in the paper is developed on the Android platform and utilizes GPS, Geo- Fencing, and SMS technologies.

The system allows parents to track the location of their children in real-time and receive emergency messages from them Further research could focus on improving the system's functionality in areas with poor network coverage or exploring alternative technologies that are not reliant on cellular networks.

Paper Title 2: A Mobile Children Safety Monitoring System Based on Android Mobile Phones

Author: Unknown

Published in: 2014 IEEE

Abstract: The paper provides a brief overview of the MCSM system and its implementation using Android mobile phones. However, it lacks detailed information on the methodology, performance evaluation, and specific datasets used. The paper describes the development of a Mobile Children Safety Monitoring (MCSM) system based on Android mobile phones.

The system aims to enhance the safety of children by preventing them from going out of the guardian's sight and detecting whether they are in safety zones. The system utilizes the hardware platform of Android mobile phones, which is widely used low cost, and easy to use. The software hand function of the system keeps the child in the guardian's eyesight, while the safety zone function monitors whether the child is in danger zones.

Paper Title 3: Design and Implementation of Child Tracking System using RFID and GPS

Author: S. S. Patil

Published in: 2018 IEEE

Abstract: The paper presents a comprehensive design and implementation of a child tracking

system using RFID and GPS technologies. The proposed system addresses the safety concerns of parents and aims to increase the confidence of children. However, the paper lacks specific details about the performance and evaluation of the system, such as the accuracy of location tracking or the effectiveness of the panic button.

Additionally, the paper does not discuss any potential limitations or challenges in implementing the system. Further research could focus on evaluating the system's performance and addressing any potential limitations to enhance its effectiveness. The system also includes a panic button for the child to use in dangerous situations. The parent can monitor the child's location using an Android application that utilizes GPS and geo-fencing

Paper Title 4: Automatic Monitoring of Children in a Classroom Environment Using Multiple

Non-Invasive Sensors

Author: S. S. Narayanan

Published in: 2012 IEEE

Abstract: This paper presents a novel application of a multi-sensor setup for monitoring children's behavior in a classroom environment. The system utilizes cameras and depth sensors to track children across multiple views effectively and for extended periods of time. The tracking of the children's activities is achieved using covariance-based descriptors on the point clouds, followed by a Kalman filter-based approach to handle noise in the depth sensors.

The paper highlights the challenges of monitoring children's activities and the potential benefits of automation tools in assisting clinicians in diagnosing behavioral abnormalities. The experiments conducted in a real classroom setting demonstrate the effectiveness of the proposed methodology. However, the paper also acknowledges some limitations and suggests future directions for improvement, such as incorporating face recognition techniques and capturing

Paper Title 5: Children Security and Tracking System using Bluetooth and GPS Technology

Author: Unknown

Published in: 2019 IEEE

Abstract: The paper provides a brief overview of the developed children security and tracking system using Bluetooth and GPS technology. The paper describes the development of a children security and tracking system using Bluetooth and GPS technology. The system utilizes an Arduino Mega microcontroller as the main controller.

However, it lacks specific details about the methodology, datasets used, and performance evaluation. Additionally, the author is unknown, which raises questions about the credibility and reliability of the information presented in the paper. Further research and analysis would be required to fully understand the effectiveness and limitations of the system.

Paper Title 6: An m-Health Monitoring System for Children with Suspected Arrhythmias

Author: E. Kyriacou, C. Pattichis, M. Pattichis, A. Jossif, L. Paraskeva, A. Konstantinides, D. Vogiatis

Published in: 2007 IEEE

Abstract: The paper presents a prototype m-Health monitoring system for children with suspected arrhythmias. It emphasizes the importance of continuous ECG monitoring for children at risk of sudden death and the challenges in deciding the proper treatment. The proposed system aims to address these challenges by providing extended ECG monitoring and automatic analysis.

However, the paper does not provide specific details on the performance or validation of the system. The paper presents the architecture and initial development steps of an m-Health monitoring system for children with suspected cardiac arrhythmias.

Paper Title 7: A Dual-Camera Based Monitoring and Alarming System for Children in Household Environment

Author: X. Li, Y. Zhang, and Y. Zhang

Published in: 2017 IEEE

Abstract: The paper presents a dual-camera based monitoring and alarming system for children in a household environment. It combines background subtraction, motion detection, and production rules to analyze children's behavior and detect potential dangers. However, the paper lacks specific experimental results and performance evaluation.

Additionally, it does not discuss the limitations or challenges of implementing the system in real world scenarios. Further research is needed to validate the system's performance and address potential limitations. The system described in the paper consists of three modules: monitoring module, anomaly judgment module, and alarming module.

Paper Title 8: Real Time Child Locating System Using ARM-7 with GPS and GSM Technology

Author: Varsha R. Chopade, Dr. P.M. Mahajan

Published in: 2018 IEEE

Abstract: The proposed child tracking system using ARM-7 with GPS and GSM technology provides a practical solution for tracking children's locations and notifying parents. The integration of voice recognition and RFID technology adds an extra layer of security and accuracy to the system. However, the paper lacks details on the specific datasets used and does not mention any research gap or limitations of the proposed system.

Further research could focus on addressing these gaps and evaluating the system's performance in real-world scenarios. The system aims to track the child's location and send a message to the parents when the child reaches the school campus. The child's voice is recorded and matched with the stored dataset to verify their presence in school.

Paper Title 9: GPS Based Vulnerable Child Tracking System

Author: Navya A, Nidhishree, Vidyashree, Vishwa, Mrs. Sadhana B

Published in: 2022 IEEE

Abstract: The proposed child tracking system using GPS and mobile GIS technology is a useful application for parents to ensure the safety and security of their children when they are away from home. The system provides real-time tracking and video monitoring capabilities, allowing parents to have peace of mind knowing their child's whereabouts and activities. The inclusion of a messaging system for communication between parents and teachers further enhances the system's functionality. The system includes features such as software hand function to monitor the child's activities and a safety sector feature to track the child's location using GPS sensors and mobile GIS.

Paper Title 10: Smart Child Safety Wearable Device

Author: Bannuru Ranjeeth, B. Srinivasa Reddy, Y. Manoj Kumar Reddy, S. Suchitra, B. Pavithra

Published in: 2020 IEEE

Abstract: Based on the abstract, the paper presents a smart child safety wearable device that integrates GPS technology, health sensors, and IoT. The proposed system aims to provide parents with the ability to track their children's location and monitor their health remotely.

However, without access to the full paper, it is difficult to provide a comprehensive analysis of the methodology, findings, and research gaps. The system is designed with both hardware and software components, including an accelerometer sensor, heart rate sensor, and a display to notify changes in coordinates. The system utilizes an ESP8266 controller, Arduino software, health sensors, and a GPS module.

Paper Title 11: Smart Child Safety Wearable Device

Author: Bannuru Ranjeeth, B. Srinivasa Reddy, Y. Manoj Kumar Reddy, S. Suchitra, B. Pavithra

Published in: 2020 JETIR

Abstract: Based on the abstract, the paper presents a smart child safety wearable device that integrates GPS technology, health sensors, and IoT. The proposed system aims to provide parents with the ability to track their children's location and monitor their health remotely. The paper proposes a smart child safety wearable device based on GPS networks. The system utilizes an ESP8266 controller, Arduino software, health sensors, and a GPS module. The ESP8266 controller is used to obtain precise location details, and the Arduino software is uploaded to the controller. The system is designed with both hardware and software components, including an accelerometer sensor, heart rate sensor, and a display to notify changes in coordinates.

Paper Title 12: Child tracking and hidden activities observation system through mobile app

Author: Mohammad Jahangir Alam, Tanjia Chowdhury, Sohrab Hossain, Shusmoy Chowdhury, Tanmoy Das

Published in: 2021 JETIR

Abstract: The proposed system aims to address the need for parents to monitor their children's activities remotely. It utilizes mobile technologies such as GPS and GIS to track the child's location and record their mobile activities. However, the paper lacks specific details on the datasets used, performance evaluation, and any identified research gaps. Further information would be needed to assess the effectiveness and limitations of the proposed system.

The proposed system consists of two sides - the child side and the parent side. The system utilizes an Android app installed on the child's mobile phone, which allows parents to monitor their children's activities remotely. The app uses GPS sensors, acceleration sensors, and mobile geographic information systems (GIS) to track the child's location. It also records the child's mobile activities, including websites visited and call history.

Paper Title 13: Child Monitoring System Using GPS Child Tracking System

Author: Sadhana B, Navya A, Nidhishree, Vidhyashree, Vishwa

Published in: 2022 JETIR

Abstract: The proposed child monitoring system using GPS and tracking technology is a relevant and practical solution to address the concerns of parents regarding their children's safety. The integration of sensors, GPS, GPRS, and cloud computing provides a comprehensive approach to monitoring a child's activities both inside and outside the school premises.

However, the paper lacks detailed information on the performance and evaluation of the system, which could be a limitation. Further research and evaluation are needed to assess the effectiveness and usability of the system in real-world scenarios. The proposed child monitoring system uses an Android phone and various sensors attached to a child's belt to

track their activities. The system utilizes GPS and GPRS modules to track the child's location and send messages to parents.

Paper Title 14: Efficient Model for Automated School Bus Safety And Security Using IOT and Cloud

Author: N Ayush Ubale, Dr Mohd Tajammul

Published in: 2022 JETIR

Abstract: A school bus security system based on RFID and GSM technologies is proposed and implemented using this system. RFID technology could be used to track students as they onboard and exit school buses.

The message is sent immediately to the parent's phone as soon as their children have successfully left or arrived at their destination, giving them peace of mind. An additional feature of the attendance checker is that it will notify parents if their children are not on board (missing). The proposed project is improved by using the Global Positioning System (GPS) as the school bus's location tracker

Paper Title 15: Child Tracking System

Author: Fairuz Rauf, Gothiswary Subramaniam & Adnan

Published in: 2018 JETIR

Abstract: Parent who live in urban area, needed to work day and night to sustain the family which causes them cannot know where their child is going during the working hour. However, with the child tracking app, parent can track and monitor their child with just a simple app. By having this child tracking system, parents can track the location of their children.

In order to avoid the kidnapping cases, the child tracking system is needed. The application will deal with the Android platform and is utilized for GPS following between different mobile devices. The application will include the route history trace where the parent track for the route their child traversed during a certain period of time. Parent also can make call from the application if any inconvenience happens when the location not found or track.

Paper Title 16: Child Monitoring System Using Android Application

Author: Netravati,

Dr. R. Savita

Published in: 2021 JETIR

Abstract: By using android Geo is combined with GSM-based SIM card into mobile to look at on Child's location. System ready to send the child's smartphone's exact location in the parent's smartphone when parent demand to test the child's location. The Client views his location on the map.

This method architecture is client server based application and mobile application. On the server-side, it uses longitude and latitude, and SMS for storing user details. On the Client-side have a box that contains a google geo-tracking device and GSM modem. When a Child is registered and logged in to the server then user details are saved to the server. This application was developed for monitoring the Child.

Paper Title 17: Child Monitoring System

Author: Roopesh, Sai Prashanth & Sivakumar

Published in: 2020 JETIR

Abstract: It is concluded that the system is reasoned that the framework is to gives youngsters security to the parents. This application is for the most part to be utilized by

guardians to find the youngster's area and guarantee their versatile utilization to identify undesirable activity on the portable. The application is utilized to follow the Child's area as well as call logs, messages and contact from their cell phone. An application at the parent side will enable guardians to send an area demand to a youngster side at that point recover the area from the demand answer and shows it on a guide. Information such as GPS coordinates and time are gathered and sent to the parent smart phone that's preregistered on the application.

Paper Title 18: An efficient child activity tracker by using context-based learning techniques.

Author: Anupama k, Deepti G, Nageena Shetty , Saritha

Published in: 2019 JETIR

Abstract: This investigation artifact enrolls a new user and stores its information in the actual time firebase database. A previous registered user may also trace her position via this App. It saves user's wandering position which may be offered to her family member directly. It is a free App and may be installed by anybody.

An Android App is quite valuable in deciding the whereabouts of the user. To identify the location of a user through Android app is very significant. Now a day, kids' uses mobile very frequently and since a very young age, the parents can take benefit of this and install the real location tracking app on their kid's mobile. This article suggests few simple actions that generate an App which is helpful in finding the location of the kids.

Paper Title 19: IoT Based Child Tracking and Safety System Author

Author: Unkown

Published in: 2023 JETIR

Abstract: The IoT-based child tracking and safety system described in the provided information seems to be a promising solution to address the increasing concerns of child safety. By utilizing GPS and GSM technology, the system enables real-time tracking and communication between parents and children.

However, further details and evaluation of the system's performance and effectiveness would be required to make a comprehensive assessment. The system aims to ensure the security and protection of children by providing real-time tracking and monitoring of their location and health. It utilizes SMS-based technology for communication, allowing parents to track their child's position on a real-time map

Paper Title 20: IoT Based Child Tracking and Safety System

Author: .Dr.S.S.Salunkhe, Sayali Patil, Shrutika Pawar, Mitali Waghmode

Published in: 2023 JETIR

Abstract: The IoT-based child tracking and safety system described in the provided information seems to be a promising solution to address the increasing concerns of child safety. By utilizing GPS, GSM, and IoT technologies, the system provides real-time tracking and monitoring capabilities, allowing parents to ensure the safety and well-being of their children. The software required includes Proteus and ARDUINO IDE.

The system implements an SMS text-enabled communication medium between the child's device and the parent using GSM mobile communication. Real-time location and heartbeat rate of the child are periodically updated in the Android application

Paper Title 21: Child Safety Device: A Prototype Model for Ensuring Security of Children

Author: C. Deenadayalan, M. Murli, L.R. Baanupriya

Published in: 2012 JETIR

Abstract: The Child Safety Device prototype model appears to be a promising solution for ensuring the security of children. The use of RFID technology, GSM communication, and voice recognition provides a reliable and user-friendly system for parents to monitor their children's safety. However, more information is needed regarding the datasets used and any potential research gaps in the existing literature

The proposed system utilizes RFID technology for tracking children and ensuring their safety. It includes a child device with a microcontroller, GSM module, voice recognition module, RFID tag, button, and IoT smartphone for GPS and camera. The parent device communicates with the child device through an Android application called Guardian App.

Paper Title 22: Mobile Tracking System Web Application

Author: Vinita, Himanshu Kaushik

Published in: 2018 JETIR

Abstract: The mobile tracking system web application seems to provide a comprehensive solution for tracking and monitoring mobile devices. The use of a user-friendly web interface and the ability to access the phone's hardware remotely are notable features. However, the performance and specific datasets used in the system are not mentioned, which limits a comprehensive evaluation of the application.

The mobile tracking system web application utilizes a user-friendly web interface and an Android application with root permissions to track and store data from the user's phone and other applications.

Paper Title 23: Enhancing Child Safety and Security in Transportation Systems

Author: Prajakta Dhamanskar, Blewett Dmonte, Russel Rodricks, Damian Fargose

Published in: 2019 JETIR

Abstract: The citations provide a range of proposed systems and technologies to enhance child safety and security in transportation systems. However, there is a lack of specific details regarding the performance and effectiveness of these systems.

Further research and valuation are needed to determine the practicality and impact of these proposed solutions. The proposed system utilizes various technologies such as GPS, GSM, RFID, biometric identification, and wearable devices to enhance child safety and security in transportation systems.

Paper Title 24: Child Monitoring System Using Android App

Author: P. Jyothsna, S.V.S. Ganga Devi

Published in: 2022 JETIR

Abstract: The child monitoring system described in the paper addresses the concern of parents who are unable to physically monitor their children due to various reasons. The app provides a way for parents to track their child's activities and location using their own mobile phone.

It emphasizes the security and privacy of the child by operating in a hidden manner on the child's smartphone. The system allows parents to monitor their child's activities remotely and view details such as call history, browsing history, and location

Paper Title 25: Child Monitoring System Using PHP and MySQL

Author: Vivek C Shet, Mr.Srinivasulu

Published in: 2022 JETIR

Abstract: The child monitoring system described in the paper aims to provide a simple and easily accessible platform for parents to view and upload complaints about missing children. The system allows parents to add their children's information and update their status.

Police can also log in to view and update the details of missing children. The child monitoring system is implemented using PHP and a MySQL database. The system offers two account options: one for parents and one for children. Parents and children need to register and log in to use the system. The system utilizes a three-layer architecture: presentation layer, business layer, and database layer.

Paper Title 26: Design and Implementation of Child Safety Using Arduino

Author: Dr. E. Kosalendra, V. Lohith, Mota Himaja Reddy, Maddikari Jagadeesh, Erikambattu Lakshmipathi, Narasanna Gari Kokila

Published in: 2023 JETIR

Abstract: Based on the provided information, it can be observed that the study focuses on designing and implementing a child safety system using Arduino and various modules. The system aims to track the child's location, notify parents about their child's whereabouts, and monitor their entry and exit from the school using RFID technology.

However, further details about the performance, datasets used, and research gap are not provided in the given information. The GPS module tracks the child's location, the GSM module transmits the location to the parent's phone, and the RFID reader is used to track the child's entry and exit from the school.

Paper Title 27: An efficient child activity tracker by using context-based learning techniques

Author: Anupama K, Deepti G, Nageena Shetty, Saritha

Published in: 2019 JEITR

Abstract: The paper addresses the concern of parents wanting to monitor and control their children's use of mobile devices. It highlights the importance of finding a balance between privacy and flexibility while providing an efficient tracking solution. The proposed application aims to assist parents in ensuring the safety and responsible use of mobile devices by their children.

The paper proposes the development of an Android application that allows parents to track and monitor their children's mobile devices. The application enables parents to view calls, texts, multimedia messages, and the location of their children. It also allows parents to set up alerts for when their children go outside approved geographical zones and block calls or messages from specified numbers.

Paper Title 28: Child Safety Using IoT

Author: Khushalsing Rajput, Ankur Chavan, Omkar Tanawade, Swapnil Sonawane

Published in: 2019 JETIR

Abstract: The proposed system addresses the need for child safety in crowded areas and aims to provide parents with real-time information about their child's location and surroundings.

The use of GPS and GSM technology, along with SMS communication, is intended to overcome the limitations of Wi-Fi and Bluetooth in terms of reliability. The system is designed to provide real-time location tracking, temperature, UV radiation index, and an alarm buzzer for the child's surroundings. The paper describes the basic design concept and functionality of the system.

2.3. References

1) A Sai Aishwarya et.al, "IOT Based Smart School Bus Monitoring and Notification System", International Journal of Engineering Research & Technology (IJERT) – 2020.

- 2) Nada Abdul Al-Balushi et.al, "Transport Safety Mechanism of School Children Using IOT based Smart System", Fourth Middle East College Student Research Conference, Muscat, Sultanate of Oman (2020).
- 3) Dr. M. V. Vyawahare et.al, "IOT Based School Bus Monitoring and Security System", International Journal of Innovative Science and Research Technology (2019).
- 4) PalviShelke et.al, "Smart Tracking System for School Buses for Ensuring Child Security using IoT Implications and GPS Technology", International Research Journal of Engineering and Technology (IRJET) (2019).
- 5) Tun Mohamad Aqil Mohamad Fadzir et.al, "Development of School Bus Security System Based on RFID and GSM Technologies for Klang Valley Area", IEEE 5th International Conference on Smart Instrumentation, Measurement and Applications (ICSIMA 2018). International Journal of Engineering Applied Sciences and Technology, 2022 Vol. 6, Issue 10, ISSN No. 2455-2143, Pages 217-225 Published Online February 2022 in IJEAST (<http://www.ijeast.com>) 225
- 6) Judy Thyparampil Raj et.al, "IoT Based Smart School Bus Monitoring and Notification System", IEEE Region 10 Humanitarian Technology Conference (R10- HTC) (2017).
- 7) S. Nath, P. Banerjee et.al, "Arduino Based Door Unlocking System with Real Time Control," 2016 International Conference on Contemporary Computing and Informatics (IC31), India, pp. 358-362, 2016.
- 8) V. Bagyaveereswaran et.al, "Design of Low Cost Portable DAQ unit for Microbial Fuel Cells," 2017 International Conference on Innovations in Power and Advanced Computing Technologies [i-PACT2017], India, pp. 1-5, 2017.
- 9) Z. Fang, W. Chen et.al, "A RFID based Kindergarten Intelligence Security System," 2012 Ninth IEEE International Conference on e-Business Engineering, China, pp. 321-326, 2012.
- 10) S. Shah et.al, "RFID Based School Bus Tracking and Security System," 2016 IEEE International Conference on Communication and Signal Processing, India, 2016, pp. 1481-1485.
- 11) Surve P et.al, "RFID & GSM Based Child Security System," International Journal of Innovative Research in Electrical, Electronics, Instrumentation and Control Engineering, India, 2016, pp. 320-323.
- 12) Kumar P et.al, "Intelligent Multilevel Car Parking System using RFID," International Journal of Simulation Systems, Science & Technology, 2016, pp. 14.1- 14.5.
- 13) N. Hafizah et.al, "Halal Kit Identifier using Radio Frequency Identification Technology," 2016 International Conference on Computer and Communication Engineering (ICCCE), Malaysia, 2016, pp. 126-131
- 14) Cassandra Dsouza, Dhanashree Rane, Anjanette Raj, Supriya Murkar, Namita Agarwal, "Design of Child Security System", Link: <https://ieeexplore.ieee.org/document/8529432>.
- 15) Akash Moodbidri, Hamid Shahnasser, "Child Safety Wearable device" Link: <https://ieeexplore.ieee.org/document/7899531>.

- 16) Alan Yudhahutama, Aciek Ida Wuryandari," Network and Application Design for. Children Tracking and Identification System with HTML5 Asynchronous Communication" Link: <https://ieeexplore.ieee.org/document/6741498>.
- 17) R. Velayutham, M. Sabari," An Innovative approach for women and children's security-based location tracking system", Link: <https://ieeexplore.ieee.org/document/7530325>.
- 18) H. Mustafa, H. Kenn, K. Sarafian, W. Scanlon and Y. Zhang, "Mobile wearable communications [Guest Editorial]," in IEEE Wireless Communications, vol. 22, no. 1, pp. 10-11, February 2015.
- 19) K. Braam, Tsung-Ching Huang, Chin-Hui Chen, E. Montgomery, S. Vo and R. Beausoleli," Wristband Vital:
- 20) Aditi Gupta, Vibhor Hamit," Child Safety & Tracking Management System by Using GPS, Geo-Fencing & Android Application: An Analysis".Link: <https://ieeexplore.ieee.org/document/7546695>.
- 21) B. Dorsemayne, J. P. Gaulier, J. P. Wary, N. Kheir and P. Urien, "Internet of things:A Definition and Taxonomy,"Next Generation Mobile Applications,Services and Technologies, 2015 9th International Conference on,2015,p.7277.
- 22) Asmita Pawar, Pratiksha Sagare, Tejal Sasane, Kiran Shinde, "Smart Security Solution for Women and Children Safety Based on GPS using IOT", Link: <https://ijrier.com/published-papers/volume-2/issue-3/smart-security-solution-for-women-andchildren-safety-based-on-gps-using-iot.pdf>.
- 23) S. Nasrin and P. I. Radcliffe, "Novel protocol enables DIY home automation," Telecommunication Networks and Applications Conference (ATNAC), 2014 Australasian, Southbank, VIC, 2014, pp.212-216. [11] F. A. Silva, "Industrial Wireless Sensor Networks: Applications, Protocols, And Standards [Book News]," in IEEE Industrial Electronics Magazine, vol. 8, no.4, pp. 67-68, Dec.2014
- 24) [1]J.Saranya, J.Selvakumar, "Implementation of Children Tracking System on Android Mobile Terminals", International conference on Communication and Signal Processing, April 3-5, 2013, India.
- 25) Rohit N. Bhoi¹ , Dr. V. V. Shete² , S.B.Somani³, "Child Tracking System on Mobile Terminal", International Journal of Advanced Research in Computer and Communication Engineering Vol. 4, Issue 6, June 2015.
- 26) Loganathan.M¹ ,Aswathi Dileep² , Kamatchi.K³, "Child Tracking System Based on GSM",International Journal of Innovative Research in Science, Engineering and Technology, Volume 4, Special Issue 4, April 2015.
- 27) Rita H. Pawade, Dr. Arun N. Gaikwad, "Android Based Children Tracking System", International Journal of Science, Engineering and Technology Research (IJSETR), Volume 4, Issue 6, June 2015.
- 28) P.SANTHARAJI¹ , V. ANURADHA², "Design and Implementation of Children Tracking System using ARM7 on Android Mobile Terminals", International Journal of Scientific Engineering and Technology Research ISSN 2319- 8885 Vol.03,Issue.21 September-2014, Pages:4327-4332
- 29) Ms. ThaniaKumar,Athul P Ravi, AThulyaBalachandran, K C Reshma, Sruthi Suresh, "My Kid : An Android Based Child Tracking System",International Journal of New Technology and Research

30) PoojaMankar, HitaliNasare, PrachiPatle, MeenalMahadole, PranaliBorkar, Swati Gupta,
“IMPLEMENTATION OF CHILDREN TRACKING SYSTEM USING MOBILE TERMINALS”,
International Journal of Advanced Research in Computer Engineering & Technology (IJARCET)
Volume 4 Issue 1, January 2015

Literature Review:

Sr. No.	Title	Author	Year of publication	Datasets	Methodology	Findings / Performance	Research Gap	Your Observations

2.4. Conclusion

The Conclusion of Child Monitoring Systems can be valuable tools for parents and guardians to enhance the safety and well-being of their children in an increasingly digital and complex world. These systems offer a range of benefits, including improved safety, peace of mind, and the ability to address online and offline risks.

To harness the advantages of Child Monitoring Systems while mitigating their drawbacks, a balanced approach is crucial. This approach includes open communication with children about the reasons for monitoring, setting clear expectations, respecting privacy, and fostering trust within the parent-child relationship. By utilizing these systems responsibly and in conjunction with guidance and education, parents can create a supportive and secure environment for their children's growth and development in today's digital age.

Chapter 3: System Analysis

Content:

- 3.1. Introduction
- 3.2. System Design
 - 3.2.1. Introduction to System Planning
 - 3.2.2. Software Design Approach
- 3.3. Gantt Chart
- 3.4. Time Line Chart
- 3.5. Cost Estimation
 - 3.5.1. Cost Beneficial Analysis
- 3.6. Feasibility
- 3.7. Conclusion

3.1. Introduction

A child monitoring system is a software or hardware-based solution designed to track and monitor the activities, safety, and well-being of children. The system can track the location of the child using GPS, Wi-Fi, or cellular networks. This feature allows parents or caregivers to know the child's whereabouts at all times. The system design phase is where the architectural framework and key components are outlined, ensuring that it fulfills its objectives effectively and efficiently.

The system can send alerts and notifications to parents or caregivers when the child engages in certain activities or when certain events occur, such as when the child leaves a designated area or when the child's location is unknown. We have created the block diagram for the system and also the System architecture. The 0 level & 1 level DFD is use to define the data flow in the system.

3.2. System Design

3.2.1. Introduction to System Planning

System planning is the process of defining the goals and requirements of a system, determining the resources needed to achieve those goals, and creating a roadmap for implementation. It involves identifying current problems, analyzing them, and designing a solution that meets the needs of the organization or project.

System planning typically includes defining objectives, identifying constraints, developing strategies, and creating a timeline for implementation. It is an essential step in the overall system development lifecycle, as it sets the direction for the development and implementation of the system.

3.2.2. Software Design Approach

A software design approach is a method or strategy used to design software systems. It encompasses the principles, techniques, and practices that guide the design process. The goal of a software design approach is to create a high-quality software system that meets the requirements of the stakeholders while being efficient, maintainable, and scalable.

The choice of a software design approach depends on various factors, including the nature of the project, the requirements of the stakeholders, and the expertise of the development team. By selecting an appropriate design approach and following best practices, software designers can create software systems that are well-structured, easy to maintain, and robust.

Each module's development entails the fundamental waterfall model processes of planning, modelling, building, and deployment, as seen in the images

[illegible][illegible]

Logbook																



Expected Time



Actual Time

	Nov 23	Dec 24					Jan 24				Feb 24				March 24	
TASKS	W4	W1	W2	W3	W4	W5	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2
CHP 1 Introduction																
CHP 2 Literature Survey																
CHP 3 Scope Of Project																
CHP 4 Methodology																
CHP 5 Design, Working and Processes																
CHP 6 Results and Application																
CHP 7 Conclusions and Future																
LOG																



Expected Time



Actual Time

3.5. Cost Estimation

Evaluating the economic viability of a "Child Monitoring System" project involves a comprehensive consideration of both costs and benefits. Development costs encompass personnel salaries, software development tools, and necessary infrastructure expenses, including the development of user interfaces and backend systems. Ongoing operational costs include maintenance, customer support, and marketing efforts to promote the system. Infrastructure expenditures such as hardware for monitoring devices, cloud services for data storage and processing, and cybersecurity tools to ensure the system's security are essential components.

The benefits of a Child Monitoring System may extend beyond financial metrics. Time savings for parents and guardians can result from real-time tracking and alerts, contributing to increased peace of mind. Improved child safety and well-being are critical outcomes, addressing concerns related to child security and providing a proactive approach to potential issues.

Moreover, the system can lead to increased productivity for parents by offering a sense of security, allowing them to focus on other responsibilities. Cost savings may also be realized by preventing potential incidents that could result in financial implications. Qualitative benefits, such as enhanced parental satisfaction and the societal impact of promoting child safety, further strengthen the project's overall value proposition.

Total Number of Weeks= $16+14=30$ weeks

Total Number of Hours= Total Number of Weeks* Number of hours per week = $30*4=120$

Cost Estimation= Total Number of hours * Cost per hour = $100*100=10,000$

Total Cost Estimation= Rs.10, 000

3.5.1. Cost Beneficial Analysis

Cost-benefit analysis (CBA) is a crucial approach for assessing the economic viability of the Child Monitoring System project by comparing its costs and benefits. Similar to the career

choice aptitude test project, development costs for the child monitoring system include personnel, software tools, and infrastructure expenses. Ongoing operational costs involve maintenance, support, and marketing efforts. Infrastructure and software expenditures, such as hardware, cloud services, and cybersecurity tools, play a vital role in ensuring the system's effectiveness and security.

On the benefits side, the child monitoring system can offer various advantages. These may include time savings for parents, improved child safety, reduced risks of potential dangers, and peace of mind for caregivers. Additionally, the system might contribute to increased productivity for parents who can focus on work or other responsibilities with the assurance that their child is safe. The project's success could lead to societal benefits by addressing concerns related to child safety and well-being.

To determine economic viability, conducting a net present value (NPV) analysis and return on investment (ROI) calculation would be essential. These financial assessments help quantify the project's profitability over time. Sensitivity analysis should also be performed to identify potential risks and uncertainties that may impact the project's success. Qualitative benefits, such as improved parental satisfaction and positive societal impacts related to child safety, further enhance the overall value proposition of the child monitoring system.

3.6. Feasibility

The feasibility of implementing a Child Monitoring System involves a comprehensive assessment of technical, economic, operational, and schedule aspects. From a technical perspective, the project needs to evaluate the available technology and resources necessary for the development and deployment of the monitoring system. This includes considering the capabilities of hardware and software components, as well as any potential challenges related to data security and privacy.

In terms of economic feasibility, the project must conduct a thorough analysis to determine if the benefits of the Child Monitoring System outweigh the associated costs. This assessment involves considering not only the initial development costs but also ongoing operational expenses, maintenance, and potential long-term benefits such as improved child safety and peace of mind for parents. Economic feasibility also entails evaluating the potential return on investment and the system's ability to provide value over time.

Operational feasibility is another critical aspect, focusing on the integration of the Child Monitoring System into existing systems and assessing user acceptance. Compatibility with existing technologies and seamless integration into daily routines are crucial for the system's success. User acceptance is particularly important to ensure that caregivers find the system user-friendly and beneficial for monitoring and ensuring the safety of their children.

Finally, schedule feasibility examines whether the project can be completed within the planned timeframe, considering potential challenges and constraints that may arise during development and implementation. Collectively, these assessments help determine the overall

feasibility of the Child Monitoring System and identify any necessary adjustments to enhance its chances of successful implementation..

3.7. Conclusion

The Conclusion of Child Monitoring Systems can be valuable tools for parents and guardians to enhance the safety and well-being of their children in an increasingly digital and complex world. In conclusion, the Child Monitoring System using Android presents a comprehensive solution for parents to ensure the safety and well-being of their children. By leveraging the capabilities of mobile technology, this system offers features such as tracking the child's location in real-time, providing access to call history and messages, and integrating with school systems for improved coordination.

Furthermore, the system's future scope includes integration with AI for intelligent monitoring and analysis, compatibility with wearable devices for enhanced tracking, and the implementation of a panic button feature for emergencies. While feasibility factors such as technical, economic, operational, and schedule considerations must be carefully evaluated, this project's objectives align with the growing need for innovative solutions in child safety. With proper planning and stakeholder engagement, the Child Monitoring System has the potential to make a significant impact in ensuring the safety and security of children in today's fast-paced world.

Chapter 4: System Design

Content:

- 4.1 Introduction
- 4.2 Block Diagram
- 4.3 System Architecture
- 4.4 Data Flow Diagram
- 4.5 Table Structure
- 4.6 State Transition Diagram
- 4.7 E-R Diagram
- 4.8 Conclusion

4.1 Introduction:

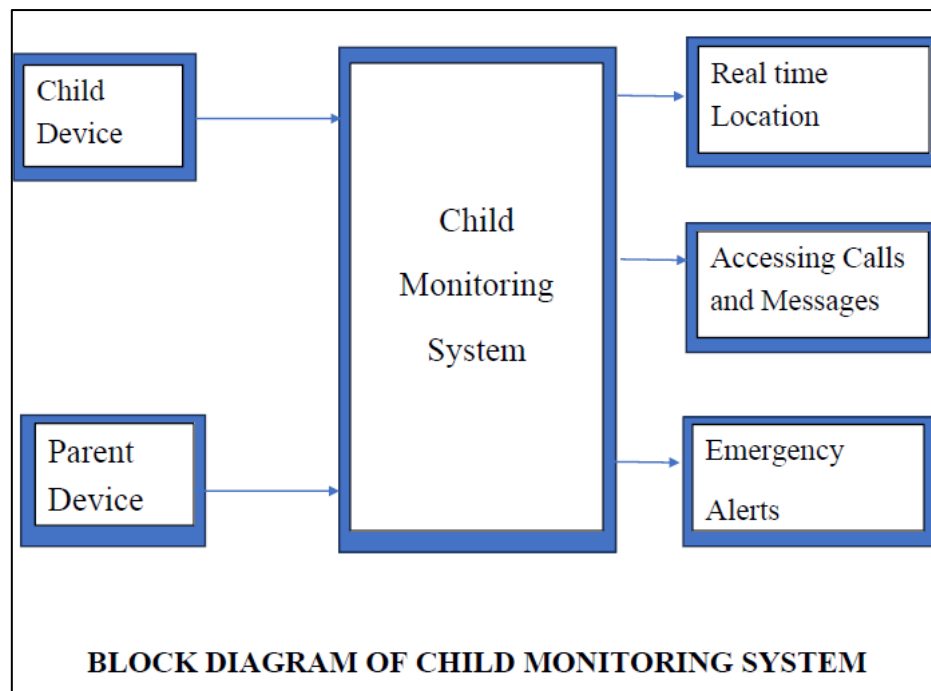
System design for the Child Monitoring System involves the meticulous planning and structuring of elements to create a robust and effective platform for ensuring child safety. The architecture of the system encompasses the overall framework, including hardware components, software applications, and communication protocols. Modules and components are designed to address specific functionalities, such as real-time monitoring, location tracking, and emergency alerts.

In this chapter we will be designing the block diagram, system architecture diagram, data flow diagram and also will plan the software design approach.

4.2 Block Diagram

A block diagram is a visual representation of a system that uses simple, labeled blocks that represent single or multiple items, entities or concepts, connected by lines to show relationships between them. An entity relationship diagram (ERD), one example of a block diagram, represents an information system by showing the relationships between people, objects, places, concepts or events within that system.

Block diagrams are used heavily in engineering and design of diagrams for electronics, hardware, software and processes. Most commonly, they represent concepts and systems in a higher level, less detailed overview. The diagrams are useful for troubleshooting technical issues



4.3 System Architecture Diagram

A system architecture diagram would be used to show the relationship between different components. Usually, they are created for systems which include hardware and software and these are represented in the diagram to show the interaction between them. However, it can also be created for web applications. For a web application the system architecture design would include components such as, database, application server, web server, internet, browser etc. Not all of these have to be included in the diagram and there are other components that can be included. Systems in a higher level, less detailed overview. The diagrams are useful for troubleshooting technical issues.

Specifications of Child Monitoring System

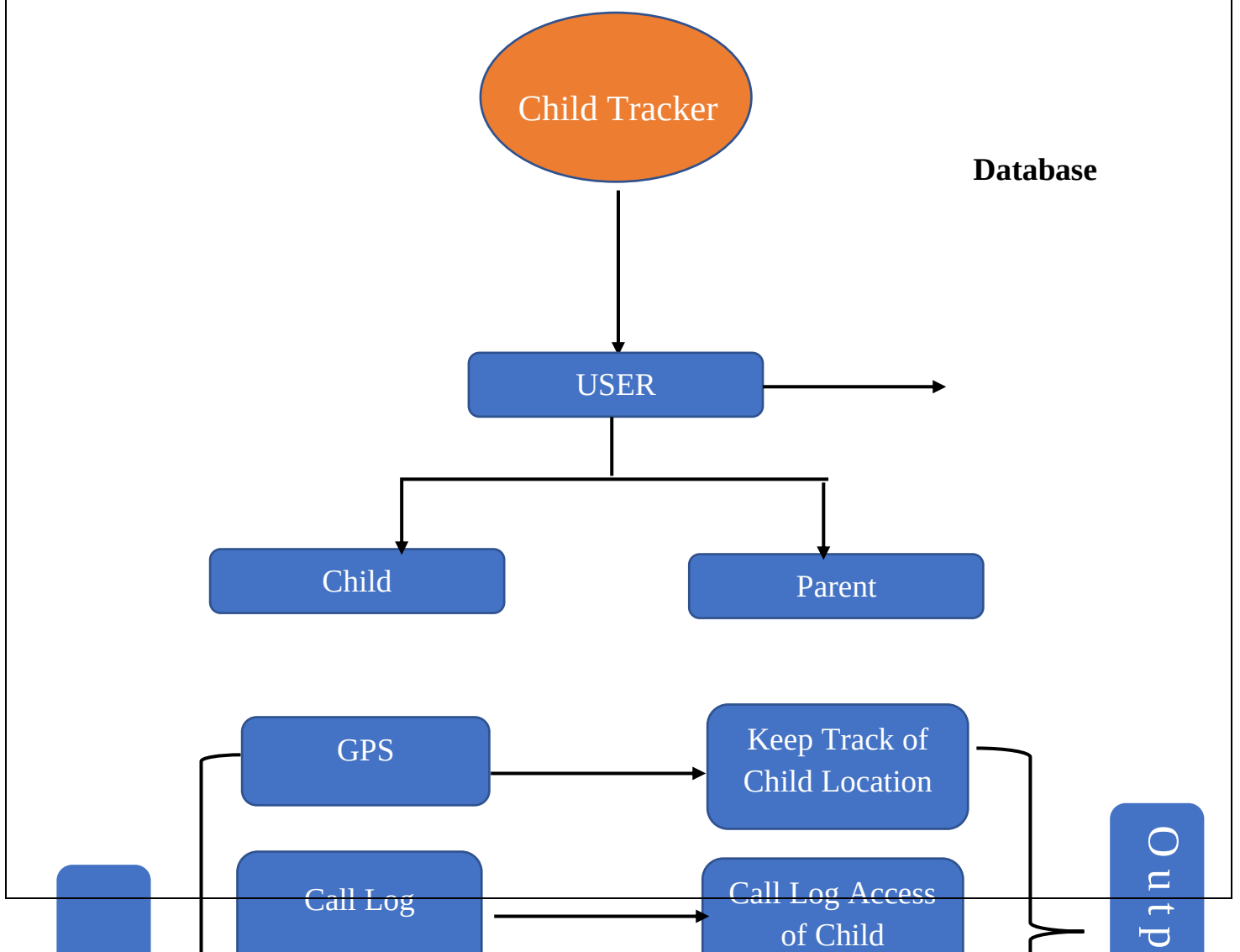
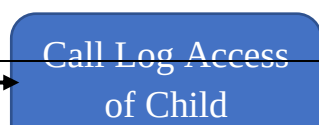
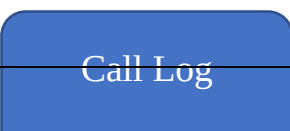
- ***Enter as a Parent or Child.***
- ***Enter the details for Login.***
- ***Parents are connected with child through unique id***
- ***Call and text message monitoring.***
- ***Logs monitoring.***
- ***Real-time location tracking.***
- ***Emergency button for Emergencies.***
- ***Parental control settings.***

- *Secure and encrypted data storage.*

Block Diagram of Child Monitoring System



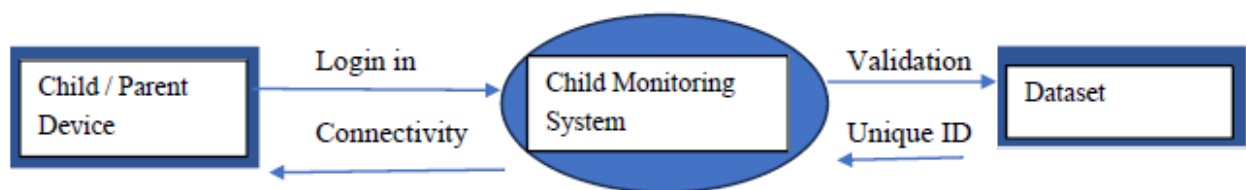
Database



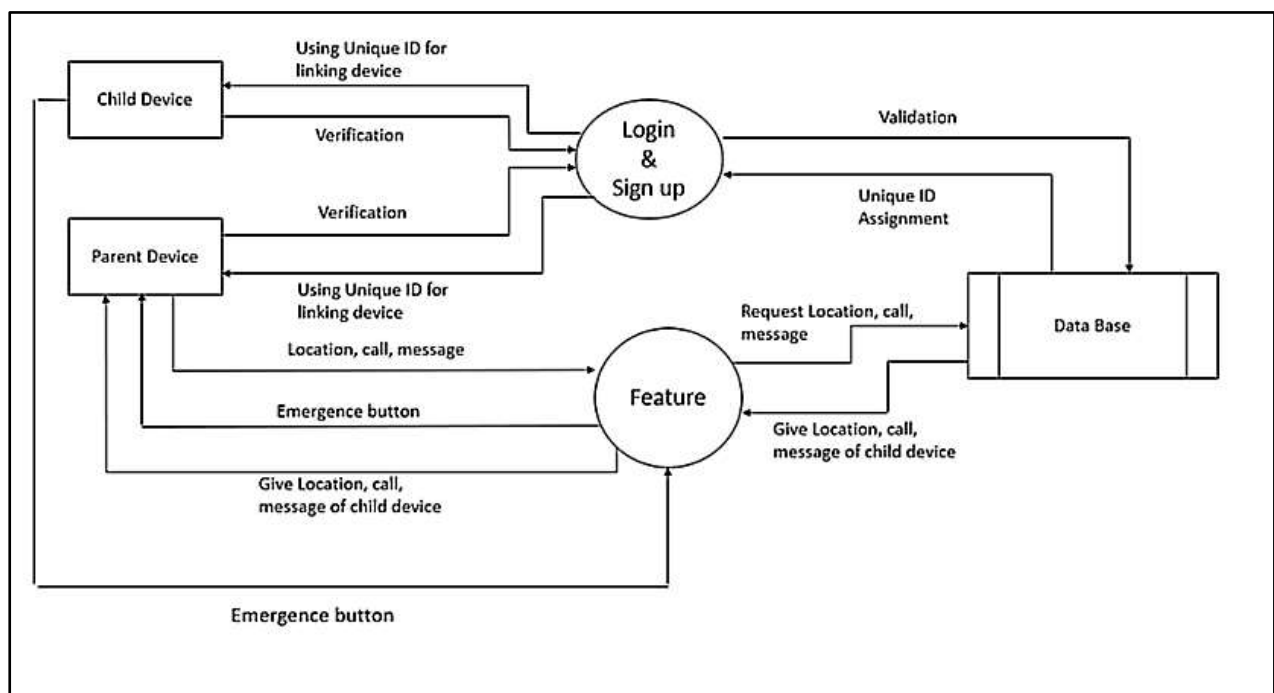
4.4 Data Flow Diagram

Also known as DFD, Data flow diagrams are used to graphically represent the flow of data in a business information system. DFD describes the processes that are involved in a system to transfer data from the input to the file storage and reports generation. Data flow diagrams can be divided into logical and physical. The logical data flow diagram describes flow of data through a system to perform certain functionality of a business. The physical data flow diagram describes the implementation of the logical data flow

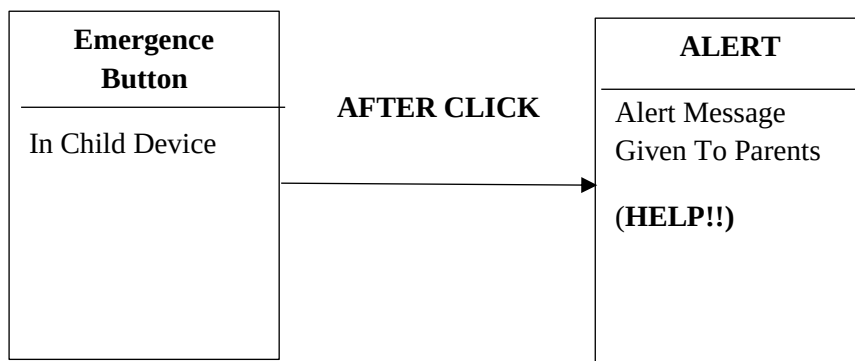
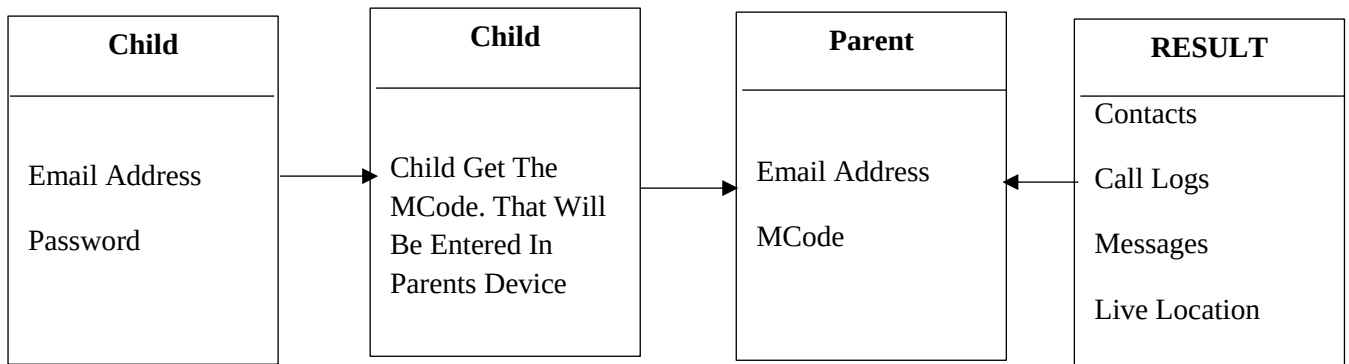
- **Level 0**



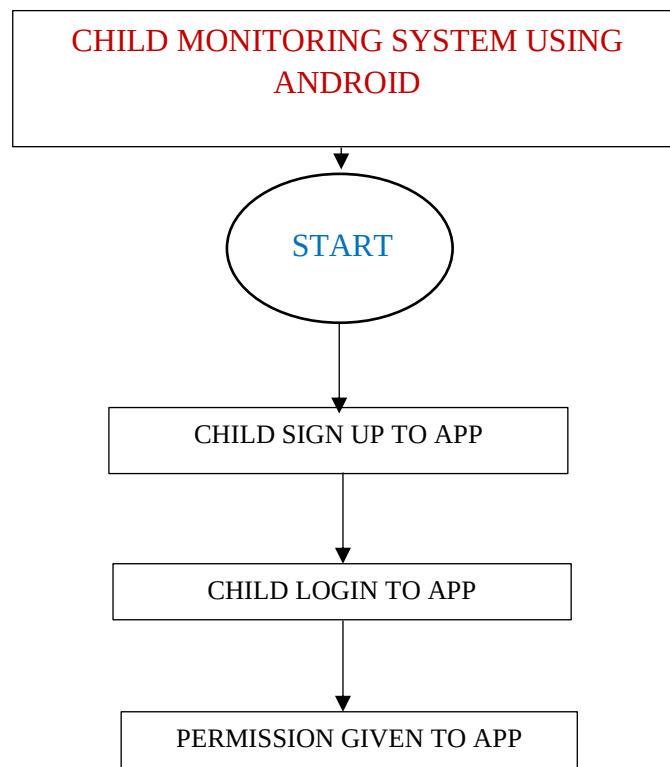
- **Level 1**

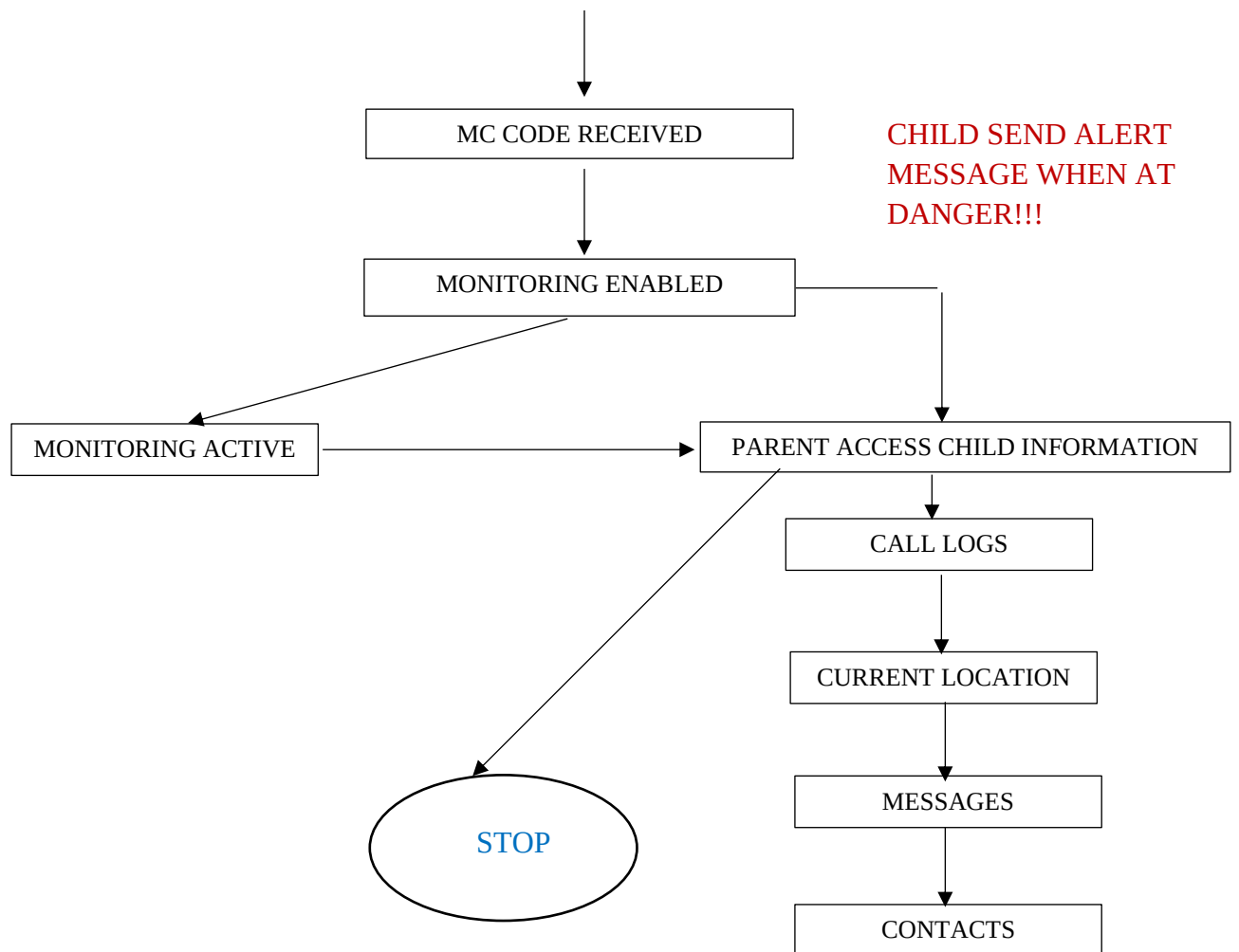


4.5 Table Structure



4.6. State Transition Diagram





4.7. E-R Diagram

LEFT TO DO

4.8. Conclusion

The analysis and design process, including the creation of an Entity-Relationship diagram, a State Transition diagram, a Block diagram, and a System Architecture, have provided valuable insights into the complexities of developing such a system. Future enhancements could include integrating AI for intelligent monitoring and analysis, as well as compatibility with wearable devices for enhanced tracking.

Overall, this project has not only deepened my understanding of Android app development but also highlighted the importance of technology in ensuring the safety and well-being of children. With proper planning and stakeholder engagement, the Child Monitoring System has the potential to make a significant impact in ensuring the safety and security of children in today's fast-paced world."

Chapter 5: Implementation

5.1. Introduction

5.2. Algorithm

5.3. Flowchart

5.4. Coding

5.5. Conclusion

5.1. Introduction

In the realm of child safety and parental supervision, our GuardianWatch app emerges as a beacon of technological advancement. As we embark on our implementation journey, we delve into a meticulous examination of algorithms, system architectures, and coding methodologies. This introduction sets the stage for a detailed exploration of the technical intricacies behind our app's development, promising a robust and intuitive child monitoring solution for the modern age.

5.2. Algorithm

Step 1: Start

Step 2: Parent opens the Child app on Child device.

Step 3: Check if the child is logged in:

- . If yes, proceed to step 6.
- . If no, go to step 4.

Step 4: Prompt the Child to either login or sign up with their credentials.

Step 5: After successful login/signup, proceed to step 6.

Step 6: Display the main dashboard of the app with options for monitoring the child's activities to Parent.

Step 7: Parent selects a specific monitoring feature (e.g., view location, view messages).

Step 8: Depending on the selected feature:

- . For viewing the child's location: Display the real-time location of the child on a map.
- . For viewing messages: Show a list of recent messages sent and received by the child.
- . For accessing contacts: Display the contacts saved on the child's device.
- . For using the alert button: Provide an option to trigger an alert from the child's device.
- . For viewing call logs: Display a list of recent calls made and received by the child.

Step 9: Offer additional options:

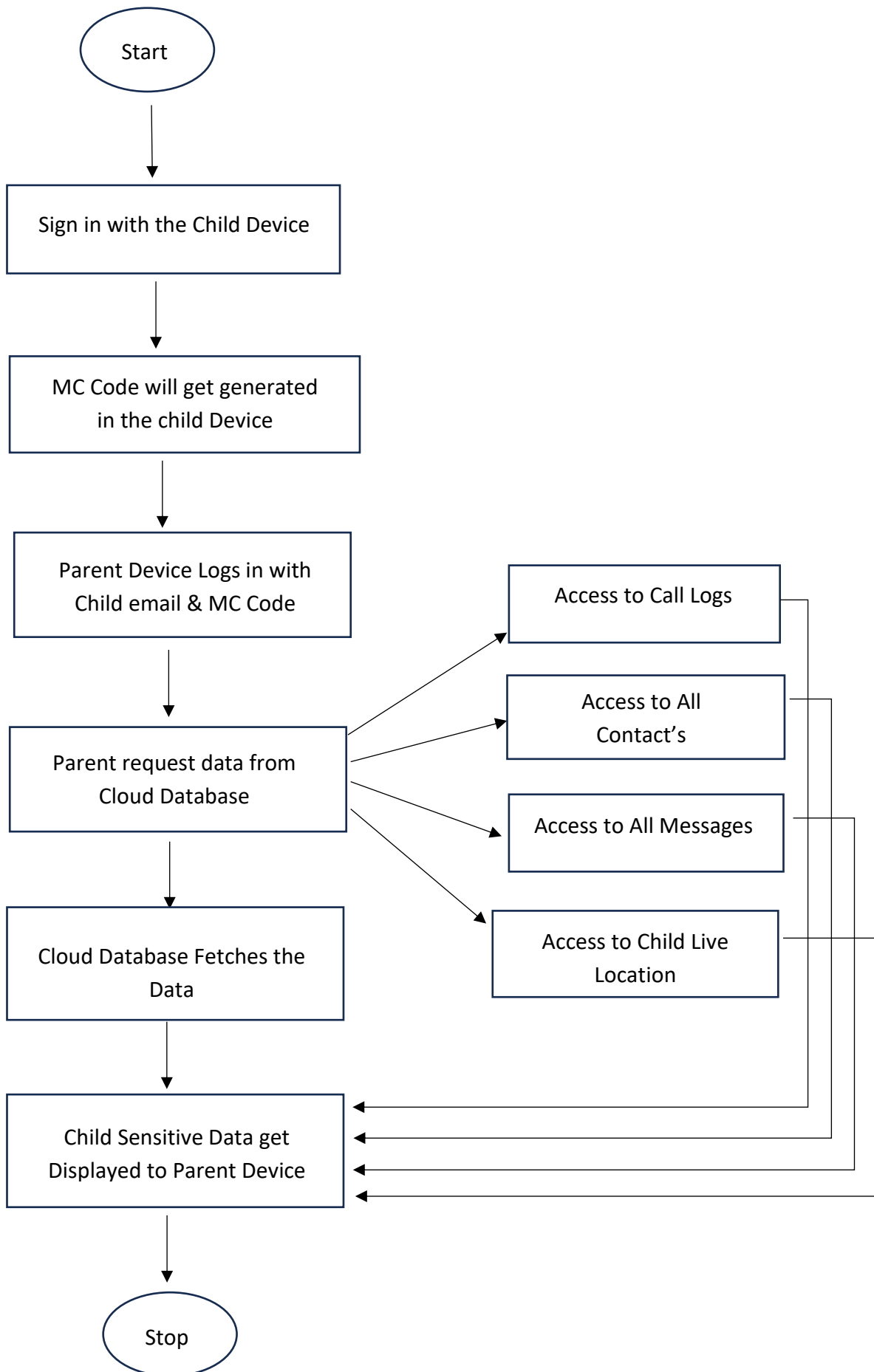
Child Device: There Is An Emergency Button When Child Is In Trouble And Child Click Button Notification Was Send to parents device (Help!!)

. Connect to Child's Device: Provide an option for the parent to connect to the child's device by entering a unique MC code generated by the child's app.

Child Can Not Logout From App. An App Always Run In A Background To Monitor Child Activities.

Step 10: End.

5.3. Flowchart



5.4. Coding

CHILD APP

Activity_Main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/and
roid"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
tools:context=".MainActivity">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:gravity="center"
    android:background="#F9F9F9">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Child Mode"
        android:fontFamily="@font/lato_black"
        android:textSize="16sp"/>
    </LinearLayout>
<androidx.core.widget.NestedScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <TextView
            android:id="@+id/permissionErrorTv"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Error: Please grant all the
permission from settings."
            android:fontFamily="@font/lato_black"
            android:textSize="18sp"
            android:textColor="@color/md_red_700"
            android:layout_marginHorizontal="12dp"
            android:layout_marginTop="30dp"
            android:visibility="gone"/>
        <androidx.cardview.widget.CardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:cardBackgroundColor="#14B7DB"
            app:cardUseCompatPadding="true"
            android:layout_marginHorizontal="12dp"
            android:layout_marginTop="14dp"
```

```
app:cardCornerRadius="16sp">
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="18dp"
        android:gravity="center_vertical">
        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:layout_marginHorizontal="8dp"
            android:fontFamily="@font/lato_black"
            android:textColor="@color/white"
            android:textSize="18sp"
            android:text="Your activities are now
being recorded"/>
        <ImageView
            android:layout_width="36dp"
            android:layout_height="36dp"

            android:src="@drawable/ic_hat_magnifier"

            android:layout_marginHorizontal="8dp"/>
    </LinearLayout>
</androidx.cardview.widget.CardView>
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Your MCode is -"
        android:textStyle="italic"
        android:fontFamily="@font/lato_black"
        android:textSize="18sp"
        android:gravity="center"
        android:layout_marginTop="20dp"/>
    <include
        android:id="@+id/mCodeView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        layout="@layout/layout_otp"
        android:layout_marginTop="18dp"/>

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Enter above MCode in your
parents device to share your data."
        android:fontFamily="@font/lato_black"
        android:textSize="18sp"
        android:layout_marginHorizontal="12dp"
        android:layout_marginTop="25dp"/>

    <LinearLayout
        android:layout_width="match_parent"
```

```

        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_marginTop="40dp">
<LinearLayout
    android:id="@+id/copyTextLayout"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_marginEnd="20dp">
<ImageView
    android:layout_width="56dp"
    android:layout_height="56dp"
    android:src="@drawable/ic_copy"/>
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Copy MCode"
    android:fontFamily="@font/lato_bold"
    android:textSize="16sp"
    android:layout_marginTop="4dp"/>
</LinearLayout>
<LinearLayout
    android:id="@+id/shareMCodeLayout"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_marginStart="20dp">
<ImageView
    android:layout_width="56dp"
    android:layout_height="56dp"
    android:src="@drawable/ic_share"/>
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Share MCode"
    android:fontFamily="@font/lato_bold"
    android:textSize="16sp"
    android:layout_marginTop="4dp"/>
</LinearLayout>
</LinearLayout>

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Note: Do not share above
MCode with unknown person."
    android:fontFamily="@font/lato"
    android:textSize="18sp"
    android:textColor="@color/md_red_700"
    android:layout_marginHorizontal="12dp"
    android:layout_marginTop="30dp"/>

<Button
    android:id="@+id/notifyParentBtn"
    android:layout_width="match_parent"

```

```

        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:layout_marginHorizontal="12dp"
        android:fontFamily="@font/lato_bold"
        android:text="Emergency (Notify Parent)"

        android:backgroundTint="@color/md_red_400"/>

</LinearLayout>

</androidx.core.widget.NestedScrollView>

</LinearLayout>

Activity_Register.xml

<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    tools:context=".auth.RegisterActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:layout_marginHorizontal="12dp"
        android:padding="4dp">

        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="250dp"
            android:layout_marginHorizontal="4dp"

            android:background="@drawable/bg_round_dark_grey"
            android:layout_marginTop="60dp">

            <ImageView
                android:layout_width="160dp"
                android:layout_height="160dp"
                android:layout_centerVertical="true"
                android:layout_centerHorizontal="true"
                android:src="@drawable/ic_round_family"/>

            </RelativeLayout>

            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Child Monitoring System"
                android:gravity="center"
                android:textSize="24sp"
                android:layout_marginTop="12dp"

```

```

        android:fontFamily="@font/lato_bold"/>

<LinearLayout
    android:id="@+id/inputFields"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:paddingHorizontal="4dp"
    android:layout_marginTop="12dp">
    <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/emailEt"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email Address">

        <EditText
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textEmailAddress" />
    </com.google.android.material.textfield.TextInputLayout
    >
    <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/passwordEt"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:layout_marginTop="4dp">

        <EditText
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPassword" />
    </com.google.android.material.textfield.TextInputLayout
    >
    <com.google.android.material.textfield.TextInputLayout
        android:id="@+id/confirmPasswordEt"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Confirm Password"
        android:layout_marginTop="4dp"
        android:visibility="visible">

        <EditText
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPassword" />
    </com.google.android.material.textfield.TextInputLayout
    >

    </LinearLayout>
    <Button
        android:id="@+id/registerBtn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="Register"
        android:fontFamily="@font/lato_bold"
        android:textSize="18sp"/>
</LinearLayout>

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:visibility="visible">
        <TextView
            android:id="@+id/newUserBtn"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center"
            android:textSize="16sp"
            android:text="Existing User? Click here to
Login "/>
        </LinearLayout>
    </LinearLayout>
</androidx.core.widget.NestedScrollView>

```

Activity_Login.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    tools:context=".auth.LoginActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:layout_marginHorizontal="12dp"
        android:padding="4dp">

        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="250dp"
            android:layout_marginHorizontal="4dp"
            android:background="@drawable/bg_round_dark_grey"
            android:layout_marginTop="60dp">
            <ImageView
                android:layout_width="160dp"
                android:layout_height="160dp"
                android:layout_centerVertical="true"
                android:layout_centerHorizontal="true"
                android:src="@drawable/ic_round_family"/>
            </RelativeLayout>
            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Child Monitoring System"
                android:gravity="center"
                android:textSize="24sp"
                android:layout_marginTop="12dp"
                android:fontFamily="@font/lato_bold"/>
        </LinearLayout>
    </LinearLayout>

```

```

        android:id="@+id/inputFields"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingHorizontal="4dp"
        android:layout_marginTop="12dp">
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/emailEt"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email Address">
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout
>
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/passwordEt"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Password"
    android:layout_marginTop="4dp">
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword" />
</com.google.android.material.textfield.TextInputLayout
>
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/mobileNumberEt"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Mobile Number"
    android:visibility="gone">

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="number" />

</com.google.android.material.textfield.TextInputLayout
>
    </LinearLayout>
    <Button
        android:id="@+id/registerBtn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="Login"
        android:fontFamily="@font/lato_bold"
        android:textSize="18sp"/>
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:visibility="visible">
        <TextView

```

```

        android:id="@+id/newUserBtn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:textSize="16sp"
        android:text="New user? Click here to Sign up"
"/>
    </LinearLayout>
</LinearLayout>
</androidx.core.widget.NestedScrollView>

```

Layout_otp.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:gravity="end"
        android:layout_marginEnd="12dp">
        <androidx.cardview.widget.CardView
            android:layout_width="50dp"
            android:layout_height="70dp"
            app:cardBackgroundColor="#D9D9D9"
            app:cardUseCompatPadding="true"
            app:cardCornerRadius="6dp">
            <TextView
                android:id="@+id/text1"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                tools:text="L"
                android:gravity="center"
                android:fontFamily="@font/lato_black"
                android:textSize="22sp"/>
        </androidx.cardview.widget.CardView>
        <androidx.cardview.widget.CardView
            android:layout_width="50dp"
            android:layout_height="70dp"
            app:cardBackgroundColor="#D9D9D9"
            app:cardUseCompatPadding="true"
            app:cardCornerRadius="6dp">
            <TextView
                android:id="@+id/text2"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                tools:text="L"
                android:gravity="center"
                android:fontFamily="@font/lato_black"
                android:textSize="22sp"/>
        </androidx.cardview.widget.CardView>
    </LinearLayout>

```



```

</androidx.cardview.widget.CardView>
<androidx.cardview.widget.CardView
    android:layout_width="50dp"
    android:layout_height="70dp"
    app:cardBackgroundColor="#D9D9D9"
    app:cardUseCompatPadding="true"
    app:cardCornerRadius="6dp">
    <TextView
        android:id="@+id/text3"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:text="L"
        android:gravity="center"
        android:fontFamily="@font/lato_black"
        android:textSize="22sp"/>
</androidx.cardview.widget.CardView>
</LinearLayout>
<LinearLayout
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:gravity="start"
    android:layout_marginStart="12dp">
<androidx.cardview.widget.CardView
    android:layout_width="50dp"
    android:layout_height="70dp"
    app:cardBackgroundColor="#D9D9D9"
    app:cardUseCompatPadding="true"
    app:cardCornerRadius="6dp">
    <TextView
        android:id="@+id/text4"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:text="L"
        android:gravity="center"
        android:fontFamily="@font/lato_black"
        android:textSize="22sp"/>
</androidx.cardview.widget.CardView>
<androidx.cardview.widget.CardView
    android:layout_width="50dp"
    android:layout_height="70dp"
    app:cardBackgroundColor="#D9D9D9"
    app:cardUseCompatPadding="true"
    app:cardCornerRadius="6dp">
    <TextView
        android:id="@+id/text5"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:text="L"
        android:gravity="center"
        android:fontFamily="@font/lato_black"
        android:textSize="22sp"/>
</androidx.cardview.widget.CardView>
<androidx.cardview.widget.CardView
    android:layout_width="50dp"
    android:layout_height="70dp"
    app:cardBackgroundColor="#D9D9D9"

```

```

    app:cardUseCompatPadding="true"
    app:cardCornerRadius="6dp">
    <TextView
        android:id="@+id/text6"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        tools:text="L"
        android:gravity="center"
        android:fontFamily="@font/lato_black"
        android:textSize="22sp"/>
</androidx.cardview.widget.CardView>
</LinearLayout>
</LinearLayout>

```

LoginActivity.kt

```

package com.expapps.monitorchildapp.auth
import android.app.ProgressDialog
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import com.expapps.monitorchildapp.MainActivity
import com.expapps.monitorchildapp.databinding.ActivityLoginBinding
import com.expapps.monitorchildapp.models.User
import com.expapps.monitorchildapp.Utils
import com.expapps.monitorchildapp.Utils.openActivity
import com.expapps.monitorchildapp.Utils.showToast
import com.google.firebase.auth.FirebaseAuth

```

```

class LoginActivity : AppCompatActivity() {
    private lateinit var binding: ActivityLoginBinding
    private lateinit var firebaseAuth: FirebaseAuth
    private lateinit var progressDialog: ProgressDialog
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
            ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)
        init()
        setLoginClick()
        setNewUserClick()
    }
    private fun init() {
        firebaseAuth = FirebaseAuth.getInstance()
        progressDialog = ProgressDialog(this)
        progressDialog.setCancelable(false)
        progressDialog.setCanceledOnTouchOutside(false)
        progressDialog.setTitle("CMS")
        progressDialog.setMessage("Registering your child...")
    }
    private fun setLoginClick() {
        binding.registerBtn.setOnClickListener {
            val email =
                binding.emailEt.editText?.text.toString().trim()
            val password =
                binding.passwordEt.editText?.text.toString().trim()

```

```

        if (!Utils.checkNotNullString(email,
password)) {
            showToast("Error: Please fill proper data")
            return@setOnClickListener
        }

        if (!Utils.checkPasswordLength(password)) {
            showToast("Error: Password should be more
than 6 letters")
        }
        val user = User(
            email = email,
            password = password
        )
        login(user)}
private fun setNewUserClick() {
    binding.newUserBtn.setOnClickListener {
        openActivity(RegisterActivity::class.java,
finishPrev = true)
    }
}
private fun login(user: User) {
    showProgress()
firebaseAuth.signInWithEmailAndPassword(user.email
?: "", user.password ?: "")
        .addOnSuccessListener {
            dismissProgress()
            showToast("Login successful")
            openActivity(MainActivity::class.java,
finishPrev = true)
        }
        .addOnFailureListener {
            dismissProgress()
            showToast("Login failed, please check
credentials")
        }
    }
private fun showProgress() {
    progressDialog.show()
}
private fun dismissProgress() {
    progressDialog.dismiss()
}
}

```

RegisterActivity.kt

```

package com.expapps.monitorchildapp.auth
import android.app.ProgressDialog
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import com.expapps.monitorchildapp.MainActivity
import com.expapps.monitorchildapp.databinding.ActivityRegis
terBinding
import com.expapps.monitorchildapp.models.User
import com.expapps.monitorchildapp.FirebaseSource
import com.expapps.monitorchildapp.Utils
import com.expapps.monitorchildapp.Utils.openActivity

```

```

import com.expapps.monitorchildapp.Utils.showToast
import com.google.firebase.auth.FirebaseAuth

```

```

class RegisterActivity : AppCompatActivity() {
    private lateinit var binding: ActivityRegisterBinding
    private lateinit var firebaseAuth: FirebaseAuth
    private lateinit var progressDialog: ProgressDialog
    private lateinit var firebaseSource: FirebaseSource
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
ActivityRegisterBinding.inflate(layoutInflater)
        setContentView(binding.root)
        init()
        setRegisterClick()
        binding.newUserBtn.setOnClickListener {
            openActivity(LoginActivity::class.java,
finishPrev = true)
        }
    }
    private fun init() {
        firebaseSource = FirebaseSource()
        firebaseAuth = FirebaseAuth.getInstance()
        progressDialog = ProgressDialog(this)
        progressDialog.setCancelable(false)
        progressDialog.setCanceledOnTouchOutside(false)
        progressDialog.setTitle("CMS")
        progressDialog.setMessage("Logging in...")
    }
    private fun setRegisterClick() {
        binding.registerBtn.setOnClickListener {
            val email =
binding.emailEt.editText?.text.toString().trim()
            val password =
binding.passwordEt.editText?.text.toString().trim()
            val confirmPassword =
binding.confirmPasswordEt.editText?.text.toString().trim
()
            if (!Utils.checkNotNullString(email,
password, confirmPassword)) {
                showToast("Error: Please fill proper data")
                return@setOnClickListener
            }
            if (!Utils.checkPasswordLength(password)) {
                showToast("Error: Password should be more
than 6 letters")
                return@setOnClickListener
            }
            if (!Utils.isAllStringsEqual(password,
confirmPassword)) {
                showToast("Error: Password must be same")
                return@setOnClickListener
            }
            val user = User(
                email = email,
                password = password
            )

```

```

        checkIfUserExist(user))}
    private fun register(user: User) {
        firebaseAuth.createUserWithEmailAndPassword(user.email ?: "", user.password ?: "")
            .addOnSuccessListener {
                dismissProgress()
                user.userId = it.user?.uid
                user.mcode =
                    Utils.getMCodeFromUserId(user.userId ?: "")
                addUser(user)
            }
            .addOnFailureListener {
                dismissProgress()
                showToast("Registration error")
            }
    }
    private fun checkIfUserExist(user: User) {
        showProgress()
        firebaseAuth.fetchSignInMethodsForEmail(user.email ?: "")
            .addOnSuccessListener {
                if (!it.signInMethods.isNullOrEmpty()) {
                    dismissProgress()
                    showToast("Error: User already exist")
                } else {
                    register(user)
                }
            }
    }
    private fun addUser(user: User) {
        showProgress()
        firebaseSource.addUser(user).observe(this) {
            if (it == true) {
                showToast("User Registered")
                openActivity(MainActivity::class.java,
                    finishPrev = true)
            } else {
                showToast("Unknown error occurred")
            }
        }
    }
    private fun showProgress() {
        progressDialog.show()
    }
    private fun dismissProgress() {
        progressDialog.dismiss()
    }
}

```

CallLogRetriver.kt

```

package com.expapps.monitorchildapp.calllogs//
CallLogRetriever.kt
import android.content.ContentResolver
import android.content.Context
import android.database.Cursor
import android.net.Uri
import android.provider.CallLog

```

```

import android.util.Log
import
com.expapps.monitorchildapp.models.CallLogEntry
class CallLogRetriever(private val context: Context) {
    fun retrieveCallLogs(): List<CallLogEntry> {
        val contentResolver: ContentResolver =
            context.contentResolver
        // Specify the columns you want to retrieve
        val projection = arrayOf(
            CallLog.Calls._ID,
            CallLog.Calls.NUMBER,
            CallLog.Calls.TYPE,
            CallLog.Calls.DATE,
            CallLog.Calls.DURATION
        )
        // Query the CallLog content provider
        val cursor: Cursor? = contentResolver.query(
            CallLog.Calls.CONTENT_URI,
            projection,
            null,
            null,
            "${CallLog.Calls.DATE} DESC" // Sort by date
            in descending order (recent first)
        )
        // Process the cursor and convert call logs to
        CallLogEntry objects
        val callLogs = mutableListOf<CallLogEntry>()
        cursor?.use {
            val idIndex =
                it.getColumnIndex(CallLog.Calls._ID)
            val numberIndex =
                it.getColumnIndex(CallLog.Calls.NUMBER)
            val typeIndex =
                it.getColumnIndex(CallLog.Calls.TYPE)
            val dateIndex =
                it.getColumnIndex(CallLog.Calls.DATE)
            val durationIndex =
                it.getColumnIndex(CallLog.Calls.DURATION)
            while (it.moveToNext()) {
                val callId = it.getString(idIndex)
                val phoneNumber = it.getString(numberIndex)
                val callType = it.getInt(typeIndex)
                val callDate = it.getLong(dateIndex)
                val callDuration = it.getLong(durationIndex)
                val callLogEntry = CallLogEntry(callId,
                    phoneNumber, callType, callDate, callDuration)
                callLogs.add(callLogEntry)
            }
        }
        // Log the retrieved call logs for debugging
        Log.d("CallLogRetriever", "Retrieved call logs:
        $callLogs")

        // Return the list of call logs
        return callLogs
    }
}

```

ContactBroadcastReceiver.kt

```
package com.expapps.monitorchildapp.contacts

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent

class ContactBroadcastReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context?, intent: Intent?) {
        // Handle the broadcast event, for example, by updating UI or performing some action
        // TODO : Upload the added contact
    }

    companion object {
        const val ACTION_CONTACT_ADDED = "com.yourapp.action.CONTACT_ADDED"

        fun getContactAddedIntent(): Intent {
            return Intent(ACTION_CONTACT_ADDED)
        }
    }
}
```

ContactObserver.kt

```
package com.expapps.monitorchildapp.contacts

import android.content.Context
import android.database.ContentObserver
import android.os.Handler
class ContactObserver(context: Context, handler: Handler) : ContentObserver(handler) {
    private val applicationContext: Context = context.applicationContext
    override fun onChange(selfChange: Boolean) {
        super.onChange(selfChange)
        // Handle the contact change, for example, by sending a broadcast
        applicationContext.sendBroadcast(ContactBroadcastReceiver.getContactAddedIntent())
    }
}
```

IUserLocationListener.kt

```
package com.expapps.monitorchildapp.location
interface IUserLocationListener {
    fun onReceivedUserLocation(lat: String?, lng: String?)
}
```

UserLocationUtils.kt

```
package com.expapps.monitorchildapp.location

import android.Manifest
import android.annotation.SuppressLint
import android.content.Context
```

```
import android.content.pm.PackageManager
import android.location.Location
import androidx.core.app.ActivityCompat
import com.google.android.gms.location.FusedLocationProviderClient
import com.google.android.gms.location.LocationRequest
import com.google.android.gms.location.LocationServices
import com.google.android.gms.tasks.CancellationTokenSource
import com.google.android.gms.tasks.Task
import org.json.JSONObject
@SuppressLint("StaticFieldLeak")
class UserLocationUtils {
    private var mFusedLocationProviderClient: FusedLocationProviderClient? = null
    private var mCurrentLocation: Location? = null
    private var currentLocationTask: Task<Location>? = null
    private var userLocationListener: IUserLocationListener? = null
    fun setUserLocationListener(userLocationListener: IUserLocationListener?) {
        this.userLocationListener = userLocationListener
    }
    fun requestLocation(context: Context) {
        mFusedLocationProviderClient = LocationServices.getFusedLocationProviderClient(context)
        onGenerateLocationToken(context)
    }
    @SuppressLint("MissingPermission")
    private fun requestCurrentLocation(context: Context) {
        currentLocationTask?.addOnSuccessListener { location: Location? ->
            mCurrentLocation = location
            userLocationListener?.onReceivedUserLocation(mCurrentLocation?.latitude.toString(), mCurrentLocation?.longitude.toString())
        }
        currentLocationTask?.addOnFailureListener { obj: Exception ->
            obj.printStackTrace()
        }
    }
    @SuppressLint("MissingPermission")
    private fun onGenerateLocationToken(context: Context) {
        if (currentLocationTask == null) {
            currentLocationTask = mFusedLocationProviderClient?.getCurrentLocation(
                LocationRequest.PRIORITY_HIGH_ACCURACY,
                CancellationTokenSource().token
            )
        }
    }
}
```

```

        requestCurrentLocation(context)
    }
}

```

MessageReceiver.kt

```

package com.expapps.monitorchildapp.messages//
MessageRetriever.kt
import android.content.ContentResolver
import android.content.Context
import android.database.Cursor
import android.net.Uri
import android.provider.Telephony
import com.expapps.monitorchildapp.models.Message
class MessageRetriever(private val context: Context) {
    fun retrieveMessages(): List<Message> {
        val uri: Uri = Uri.parse("content://sms")
        val contentResolver: ContentResolver =
            context.contentResolver
        // Specify the columns you want to retrieve
        val projection = arrayOf(
            Telephony.Sms._ID,
            Telephony.Sms.ADDRESS,
            Telephony.Sms.BODY,
            Telephony.Sms.DATE
        )
        // Query the SMS content provider
        val cursor: Cursor? = contentResolver.query(uri,
            projection, null, null, null)
        // Process the cursor and convert messages to
        Message objects
        val messages = mutableListOf<Message>()
        cursor?.use {
            val idIndex =
                it.getColumnIndex(Telephony.Sms._ID)
            val addressIndex =
                it.getColumnIndex(Telephony.Sms.ADDRESS)
            val bodyIndex =
                it.getColumnIndex(Telephony.Sms.BODY)
            val dateIndex =
                it.getColumnIndex(Telephony.Sms.DATE)
            while (it.moveToNext()) {
                val messageId = it.getString(idIndex)
                val sender = it.getString(addressIndex)
                val body = it.getString(bodyIndex)
                val timestamp = it.getLong(dateIndex)
                val message = Message(messageId, sender,
                    body, timestamp)
                messages.add(message)
            }
        }
        // Return the list of messages
        return messages
    }
}

```

CallLogEntry.kt

```

package com.expapps.monitorchildapp.models

```

```

data class CallLogEntry(
    val callId: String?,
    val phoneNumber: String?,
    val callType: Int?,
    val callDate: Long?,
    val callDuration: Long?
)

```

Contact.kt

```

package com.expapps.monitorchildapp.models

```

```

data class Contact(val id: String?, val name: String?, val
    phoneNumbers: List<String>?)

```

Locations.kt

```

package com.expapps.monitorchildapp.models

```

```

data class Locations(
    var latitude: Double? = null,
    var longitude: Double? = null
)

```

Message.kt

```

package com.expapps.monitorchildapp.models

```

```

// Message.kt
data class Message(
    val messageId: String?,
    val sender: String?,
    val body: String?,
    val timestamp: Long?
)

```

RegisteredTokens.kt

```

package com.expapps.monitorchildapp.models

```

```

data class RegisteredTokens(
    var uid: String? = "",
    var subId: String? = "",
    var token: String? = ""
)

```

User.kt

```

package com.expapps.monitorchildapp.models

```

```

data class User(
    var userId: String? = null,
    var email: String? = null,
    var password: String? = null,
    var mcode: String? = null
)

```

CollectorService.kt

```

package com.expapps.monitorchildapp.service

```

```

import android.annotation.SuppressLint

```

```

import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.content.ContentResolver
import android.content.Context
import android.content.Intent
import android.database.Cursor
import android.location.Location
import android.os.Binder
import android.os.Build
import android.os.IBinder
import android.os.Looper
import android.provider.ContactsContract
import android.util.Log
import androidx.core.app.NotificationCompat
import com.expapps.monitorchildapp.FirebaseSource
import com.expapps.monitorchildapp.MainActivity
import com.expapps.monitorchildapp.R
import
com.expapps.monitorchildapp.calllogs.CallLogRetriever
import
com.expapps.monitorchildapp.contacts.ContactObserver
import
com.expapps.monitorchildapp.location.IUserLocationLis
tener
import
com.expapps.monitorchildapp.location.UserLocationUtil
s
import
com.expapps.monitorchildapp.messages.MessageRetriev
er
import com.expapps.monitorchildapp.models.Contact
import com.expapps.monitorchildapp.models.Locations
import
com.google.android.gms.location.FusedLocationProvider
Client
import
com.google.android.gms.location.LocationCallback
import
com.google.android.gms.location.LocationRequest
import com.google.android.gms.location.LocationResult
import
com.google.android.gms.location.LocationServices
import
com.google.android.gms.tasks.CancellationTokenSource
import com.google.android.gms.tasks.Task
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import java.util.Timer
import kotlin.concurrent.timerTask

class CollectorService : Service() {
    private val CHANNEL_ID = "Main_Channel"
    private val coroutineScope =
CoroutineScope(Dispatchers.IO)
    private var contactObserver: ContactObserver? = null

```

```

    var timer: Timer? = null
    var isTimerRunning = false
    private var messageRetriever: MessageRetriever? =
null
    private var callLogRetriever: CallLogRetriever? = null
    private var fusedLocationClient:
FusedLocationProviderClient? = null
    private var locationRequest: LocationRequest? = null
    private var locationCallback: LocationCallback? = null
    private val binder = LocalBinder()
    private var firebaseSource: FirebaseSource? = null
    inner class LocalBinder : Binder() {
        fun getService(): CollectorService =
this@CollectorService
    }
    override fun onCreate() {
        super.onCreate()
        setLocationListener()
        // contactObserver = ContactObserver(this,
Handler())
        // contentResolver.registerContentObserver(
// ContactsContract.Contacts.CONTENT_URI,
// true,
// contactObserver!!
// )
    }
    private fun setLocationListener() {
        firebaseSource = FirebaseSource()
        fusedLocationClient =
LocationServices.getFusedLocationProviderClient(this)
        locationRequest = LocationRequest.create().apply {
            interval = 10000 // Update interval in
milliseconds (10 seconds)
            fastestInterval = 5000 // Fastest update interval in
milliseconds (5 seconds)
            priority =
LocationRequest.PRIORITY_HIGH_ACCURACY
        }
        locationCallback = object : LocationCallback() {
            override fun onLocationResult(locationResult:
LocationResult) {
                for (location in locationResult.locations) {
                    // Handle location updates
                    val latitude = location.latitude
                    val longitude = location.longitude

                    firebaseSource?.addCurrentLocation(Locations(latitude,
longitude))

                    Log.d("LocationUpdateService", "Updated
Latitude: $latitude, Longitude: $longitude")
                }
            }
            override fun onStartCommand(intent: Intent?, flags:
Int, startId: Int): Int {
                messageRetriever =
MessageRetriever(this@CollectorService)
                callLogRetriever =
CallLogRetriever(this@CollectorService)
                val notificationIntent = Intent(this,
MainActivity::class.java)

```

```

        val pendingIntent =
            PendingIntent.getActivity(this, 0,
            notificationIntent, PendingIntent.FLAG_MUTABLE)
            createNotificationChannel()
            val notification = NotificationCompat.Builder(this,
            CHANNEL_ID)
                .setContentTitle("Child Monitoring")
                .setContentText("Data collector is running in
background")
                .setSmallIcon(R.drawable.ic_hat_magnifier)
                .setContentIntent(pendingIntent)
                .build()
            startForeground(1, notification)
            startTimer()
            startLocationUpdates()

            return START_STICKY
        }
        override fun onBind(p0: Intent?): IBinder? {
            return binder
        }
        private fun startLocationUpdates() {
            try {
                if (locationCallback != null && locationRequest
!= null) {
                    fusedLocationClient?.requestLocationUpdates(locationR
equest, locationCallback, Looper.getMainLooper())
                }
            } catch (securityException: SecurityException) {
                Log.e("LocationUpdateService", "Location
permission not granted")
            }
        }
        private fun stopLocationUpdates() {
            fusedLocationClient?.removeLocationUpdates(locationC
allback)
        }
        private fun createNotificationChannel() {
            if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.O) {
                val serviceChannel = NotificationChannel(
                    CHANNEL_ID,
                    "Collector Service",
                    NotificationManager.IMPORTANCE_DEFAULT
                )
                val manager = getSystemService(
                    NotificationManager::class.java
                )
                manager.createNotificationChannel(serviceChannel)
            }
        }
        private fun fetchAllContacts() {
            coroutineScope.launch {
                val contacts = getAllContacts()
                Log.d("TAG", "fetchAllContacts: $contacts")
                firebaseSource?.addContacts(contacts)
            }
            coroutineScope.launch {

```

```

                val messages =
                    messageRetriever?.retrieveMessages()
                    Log.d("TAG", "fetchAllContacts: $messages")
                    messages?.let { firebaseSource?.addMessages(it)
                }
            }
            coroutineScope.launch {
                val callLogs =
                    callLogRetriever?.retrieveCallLogs()
                    Log.d("TAG", "fetchAllContacts: $callLogs")
                    callLogs?.let { firebaseSource?.addCallLogs(it) }
            }
            CoroutineScope(Dispatchers.Main).launch {
                //setLocationListener()
                startLocationUpdates()
            }
        }
        private fun getAllContacts(): List<Contact> {
            val contacts = mutableListOf<Contact>()
            val contentResolver: ContentResolver =
contentResolver
            val cursor: Cursor? = contentResolver.query(
                ContactsContract.Contacts.CONTENT_URI,
                null,
                null,
                null,
                null
            )
            if (cursor != null && cursor.count > 0) {
                val idColumnIndex =
cursor.getColumnIndex(ContactsContract.Contacts._ID)
                val nameColumnIndex =
cursor.getColumnIndex(ContactsContract.Contacts.DISPLAY_NAME)

                while (cursor.moveToNext()) {
                    val contactId =
cursor.getString(idColumnIndex)
                    val name =
cursor.getString(nameColumnIndex)
                    val phoneNumbers =
getContactPhoneNumbers(contactId)
                    contacts.add(Contact(contactId, name,
phoneNumbers))
                }
            }
            cursor?.close()
            return contacts
        }
        private fun getContactPhoneNumbers(contactId:
String): List<String> {
            val phoneNumbers = mutableListOf<String>()
            val phoneCursor: Cursor? = contentResolver.query(
                ContactsContract.CommonDataKinds.Phone.CONTENT
_URI,
                null,
                ContactsContract.CommonDataKinds.Phone.CONTACT
_ID + " = ?",
                arrayOf(contactId),

```

```

        null
    )
    if (phoneCursor != null && phoneCursor.count > 0)
    {
        val phoneNumberColumnIndex =
        phoneCursor.getColumnIndex(ContactsContract.CommonDataKinds.Phone.NUMBER)
        while (phoneCursor.moveToNext()) {
            val phoneNumber =
            phoneCursor.getString(phoneNumberColumnIndex)
            phoneNumbers.add(phoneNumber) } }
        phoneCursor?.close()
        return phoneNumbers
    }
    private fun startTimer() {
        if (!isTimerRunning) {
            timer = Timer()
            timer?.schedule(
                timerTask {
                    fetchAllContacts()
                }, 0, 60 * 1000
            )
            isTimerRunning = true
        }
    }
    private fun stopTimer() {
        if (isTimerRunning) {
            timer?.cancel()
            isTimerRunning = false
        }
    }
    override fun onDestroy() {
        super.onDestroy()
        stopTimer()
        stopLocationUpdates()
        // contactObserver?.let {
        contentResolver.unregisterContentObserver(it) }
    }
}

```

ChildApp.kt

```

package com.expapps.monitorchildapp

import android.app.Application
import com.onesignal.OneSignal
import com.onesignal.debug.LogLevel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

const val ONESIGNAL_APP_ID = "b0b06515-ee7d-49a3-aa26-22b89344f74a"
class ChildApp : Application() {
    override fun onCreate() {
        super.onCreate()
        OneSignal.Debug.logLevel = LogLevel.VERBOSE
        OneSignal.initWithContext(this,
        ONESIGNAL_APP_ID)
    }
}

```

```

        CoroutineScope(Dispatchers.IO).launch {
        OneSignal.Notifications.requestPermission(true)
        }
        OneSignal.User.pushSubscription.optIn()}}

```

Constants.kt

```

package com.expapps.monitorchildapp

```

```

object Constants {
    const val USER_ID = "USER_ID"
}

```

FirebaseSource.kt

```

package com.expapps.monitorchildapp

```

```

import android.util.Log
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import com.expapps.monitorchildapp.models.CallLogEntry
import com.expapps.monitorchildapp.models.Contact
import com.expapps.monitorchildapp.models.Locations
import com.expapps.monitorchildapp.models.Message
import com.expapps.monitorchildapp.models.RegisteredTokens
import com.expapps.monitorchildapp.models.User
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.database.DataSnapshot
import com.google.firebase.database.DatabaseError
import com.google.firebase.database.FirebaseDatabase
import com.google.firebase.database.ValueEventListener

```

```

class FirebaseSource {
    private var firebaseAuth = FirebaseAuth.getInstance()
    private var firebaseDatabase =
    FirebaseDatabase.getInstance()
    private var databaseReference =
    firebaseDatabase.reference
    fun getUserList(): LiveData<ArrayList<User>?> {
        val users = MutableLiveData<ArrayList<User>?>()
        databaseReference.child("Users")
        .addListenerForSingleValueEvent(object :
        ValueEventListener {
            val userArray = ArrayList<User>()
            override fun onDataChange(snapshot:
            DataSnapshot) {
                snapshot.children.forEach {
                    val user = it.getValue(User::class.java)
                    user?.let { u ->
                        userArray.add(u)
                    }
                }
                users.value = userArray
            }

            override fun onCancelled(error: DatabaseError)
            {

```



```

        users.value = null
    }
    })
    return users
}
fun addUser(user: User): LiveData<Boolean?> {
    val isSuccess = MutableLiveData<Boolean>()
    databaseReference.child("Users")
        .child(user.userId ?: "")
        .setValue(user)
        .addOnSuccessListener {
            addMCode(user.email ?: "",
Utils.getMCodeFromUserId(user.userId ?: ""),
user.userId ?: "")
            isSuccess.value = true
        }
        .addOnFailureListener {
            isSuccess.value = false
        }
    return isSuccess
}
fun addMCode(email: String, code: String, uid: String)
{
    databaseReference.child("UsersMCode")
        .updateChildren(
            hashMapOf<String, Any>(<
                Pair(
                    Utils.getEmailFromEmailId(email), uid
                )
            )
        )
        .addOnSuccessListener {}
        .addOnFailureListener {}
    }
    fun getMCodeByEmail(email: String):
LiveData<String?> {
        val isSuccess = MutableLiveData<String>()
        val emailStr = Utils.getEmailFromEmailId(email)
        databaseReference.child("UsersMCode/$emailStr")
            .addListenerForSingleValueEvent(object :
ValueEventListener {
                override fun onDataChange(snapshot:
DataSnapshot) {
                    if (snapshot.exists()) {
                        val child = snapshot.value.toString()
                        isSuccess.value =
Utils.getMCodeFromUserId(child)
                    } else {
                        isSuccess.value = null
                    }
                }
            })
        override fun onCancelled(error: DatabaseError)
{
            isSuccess.value = null
        }
    })
    return isSuccess
}

```

```

    fun addContacts(contact: List<Contact>) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/contacts")
            .setValue(contact)
    }
    fun addCallLogs(callLogEntry: List<CallLogEntry>) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/callLogs")
            .setValue(callLogEntry)
    }
    fun addMessages(messages: List<Message>) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/messages")
            .setValue(messages)
    }
    fun addCurrentLocation(locations: Locations) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/locations")
            .setValue(locations)
    }
    fun getRegisteredTokensForCurrentUser(uid: String):
LiveData<ArrayList<RegisteredTokens>?> {
        val _registeredTokens =
MutableLiveData<ArrayList<RegisteredTokens>?>()
        databaseReference.child("RegisteredParents/$uid")
            .addListenerForSingleValueEvent(object :
ValueEventListener {
                override fun onDataChange(snapshot:
DataSnapshot) {
                    val registeredParents =
ArrayList<RegisteredTokens>()
                    if (snapshot.exists()) {
                        snapshot.children.forEach {
                            val parent =
it.getValue(RegisteredTokens::class.java)
                            if (parent != null) {
                                registeredParents.add(parent)
                            }
                        }
                    }
                    _registeredTokens.value = registeredParents
                }
            })
        override fun onCancelled(error: DatabaseError)
{
            _registeredTokens.value = null
        }
    })
    return _registeredTokens
}
fun isLoggedIn(): Boolean {
    return firebaseAuth.currentUser != null
}
}

```

MainActivity.kt

```

package com.expapps.monitorchildapp

import android.app.ActivityManager

```

```

import android.content.ClipData
import android.content.ClipboardManager
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.os.Bundle
import android.view.View
import
androidx.activity.result.contract.ActivityResultContracts
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.ContextCompat
import com.expapps.monitorchildapp.Utils.openActivity
import com.expapps.monitorchildapp.Utils.showToast
import
com.expapps.monitorchildapp.auth.RegisterActivity
import
com.expapps.monitorchildapp.databinding.ActivityMain
Binding
import
com.expapps.monitorchildapp.models.RegisteredTokens
import
com.expapps.monitorchildapp.service.CollectorService
import com.google.firebase.auth.FirebaseAuth
import java.util.Locale
class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding
    private lateinit var firebaseSource: FirebaseSource
    private lateinit var firebaseAuth: FirebaseAuth
    private var mCode = ""
    private var registeredTokens =
ArrayList<RegisteredTokens>()
    val permissions = arrayOf(
android.Manifest.permission.ACCESS_FINE_LOCATION,
android.Manifest.permission.ACCESS_COARSE_LOCATION,
        android.Manifest.permission.READ_SMS,
android.Manifest.permission.READ_CALL_LOG,
android.Manifest.permission.READ_CONTACTS,
android.Manifest.permission.ACCESS_BACKGROUND_LOCATION,
        // Add more permissions as needed
    )
    private val requestPermissionLauncher =
registerForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) { p ->
        val allPermissionsGranted = p.all { it.value }
        if (allPermissionsGranted) {
            // Permission granted, proceed with your logic
            startService()
            binding.permissionErrorTv.visibility =
View.GONE
        } else {
            binding.permissionErrorTv.visibility =
View.VISIBLE
            // Permission denied, handle accordingly (e.g.,
show a message or disable functionality)
        }
    }
}

```

```

    }
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)
        init()
        binding.copyTextLayout.setOnClickListener {
            copyToClipboard()
        }
        binding.shareMCodeLayout.setOnClickListener {
            shareMCode()
        }
        binding.notifyParentBtn.setOnClickListener {
            notifyParents()
        }
    }
    private fun notifyParents() {
        val bearerToken =
"NjQwZGQ5YzItNTY4OS00NTVILTgxZTYtOGZkYjhlNjBkYTc1"
        val appId = "b0b06515-ee7d-49a3-aa26-22b89344f74a"
        val map = HashMap<String, Any>()
        val tokens = ArrayList<String>()
        val subs = ArrayList<String>()
        registeredTokens.forEach {
            tokens.add(it.token ?: "")
            subs.add(it.subId ?: "")
        }
        map["app_id"] = appId
        map["include_subscription_ids"] = subs
        // map["include_aliases"] =
        // hashMapOf("external_id" to subs.first())
        val content = "Your child is in emergency"
        val heading = "Child emergency alert"
        map["contents"] = hashMapOf("en" to content)
        map["headings"] = hashMapOf("en" to heading)
        NetworkController.sendNotification(map,
bearerToken)
    }
    override fun onStart() {
        super.onStart()
        checkLoggedInStatus()
    }
    private fun init() {
        firebaseSource = FirebaseSource()
        firebaseAuth = FirebaseAuth.getInstance()
    }
    private fun getMCode() {
        val email =
Utils.getEmailFromEmailId(firebaseAuth.currentUser?.email ?: "")
        firebaseSource.getMCodeByEmail(email).observe(this) {
            if (!it.isNullOrEmpty()) {
                val codeInCaps =
it.toUpperCase(Locale.ROOT)
                mCode = codeInCaps
            }
        }
    }
}

```

```

        if (codeInCaps.length == 6) {
            binding.mCodeView.text1.text =
codeInCaps[0].toString()
            binding.mCodeView.text2.text =
codeInCaps[1].toString()
            binding.mCodeView.text3.text =
codeInCaps[2].toString()
            binding.mCodeView.text4.text =
codeInCaps[3].toString()
            binding.mCodeView.text5.text =
codeInCaps[4].toString()
            binding.mCodeView.text6.text =
codeInCaps[5].toString()}}}
        private fun copyToClipboard() {
            val clipboard: ClipboardManager =
getSystemService(Context.CLIPBOARD_SERVICE) as
ClipboardManager
            val clip = ClipData.newPlainText("Copied Text",
mCode)
            clipboard.setPrimaryClip(clip)
            showToast("Copied to clipboard")
        }
        private fun shareMCode() {
            try {
                val intent = Intent(Intent.ACTION_SEND)
                val shareBody = "Your Child monitoring system
MCode is: $mCode"
                intent.type = "text/plain"
                intent.putExtra(
                    Intent.EXTRA_SUBJECT, ""
                )
                intent.putExtra(Intent.EXTRA_TEXT,
shareBody)
                startActivity(Intent.createChooser(intent, "Share
MCode to"))
            } catch (e: Exception) {
                e.printStackTrace()
            }
        }
        private fun fetchData() {
            getMCode()
            runService()
            getRegisteredTokens()
        }
        private fun getRegisteredTokens() {
            firebaseSource.getRegisteredTokensForCurrentUser(fire
baseAuth.currentUser?.uid ?: "")
                .observe(this) {
                    if (!it.isNullOrEmpty()) {
                        registeredTokens = it}}}
        private fun checkLoggedInStatus() {
            if(!firebaseSource.isLoggedIn()) {
                openActivity(RegisterActivity::class.java,
finishPrev = true)
            } else {
                fetchData()}}
        private fun startService() {
            try {

```

```

                if (!isServiceRunning(this,
CollectorService::class.java)) {
                    val myServiceIntent = Intent(this,
CollectorService::class.java)
                    ContextCompat.startForegroundService(this,
myServiceIntent)
                }
            } catch (e: Exception) {
                e.printStackTrace()}}
        private fun isServiceRunning(context: Context,
serviceClass: Class<*>): Boolean {
            val activityManager =
context.getSystemService(Context.ACTIVITY_SERVIC
E) as ActivityManager?
            activityManager?.let {
                // Get a list of running services
                for (service in
it.getRunningServices(Integer.MAX_VALUE)) {
                    if (serviceClass.name ==
service.service.className) {
                        // The service is running
                        return true}}} // The service is not running
                return false
            }
        }
        private fun runService() {
            if (checkPermission()) {
                // Permission is already granted, proceed with
your logic
                startService()
            } else {
                // Permission has not been granted, request it
                requestPermission()
            }
        }
        private fun checkPermission(): Boolean {
            for (permission in permissions) {
                if (ContextCompat.checkSelfPermission(this,
permission) !=
PackageManager.PERMISSION_GRANTED) {
                    return false
                }
            }
            return true
        }
        private fun requestPermission() {
            requestPermissionLauncher.launch(permissions)
        }
    }
}

```

NetworkController.kt

```

package com.expapps.monitorchildapp

import com.google.gson.Gson
import okhttp3.Call
import okhttp3.Callback
import import
import okhttp3.MediaType.Companion.toMediaTypeOrNull
import import okhttp3.OkHttpClient

```

```

import okhttp3.Request
import okhttp3.RequestBody
import okhttp3.Response
import java.io.IOException
object NetworkController {
    fun sendNotification(map: HashMap<String, Any>,
bearerToken: String) {
        try {
            // map["include_subscription_ids"] =
            arrayOf("f1a910fc-355d-4609-b208-1efa5f0dba30")
            map["target_channel"] = "push"
            val json = Gson().toJson(map)
            val client = OkHttpClient()
            val mediaType =
"application/json".toMediaTypeOrNull()
            val body = RequestBody.create(mediaType, json)
            val request = Request.Builder()

.url("https://onesignal.com/api/v1/notifications")
            .post(body)
            .addHeader("accept", "application/json")
            .addHeader(
                "Authorization",
                "Basic $bearerToken"
            )
            .addHeader("content-type", "application/json")
            .build()
            client.newCall(request).enqueue(object : Callback
{
                override fun onResponse(call: Call, response:
Response) {}
                override fun onFailure(call: Call, e:
IOException) {}
            })
        } catch (e: Exception) {
            e.printStackTrace()
        }
    }
}

```

Utils.kt

```

package com.expapps.monitorchildapp

import android.content.Context
import android.content.Intent
import android.widget.Toast
import androidx.fragment.app.FragmentActivity
object Utils {
    fun checkEmptyOrNullString(vararg strings: String?):
Boolean {
        if (strings.contains(null) || strings.contains("null") ||
strings.contains("")) {
            return false
        }
        return true
    }
    fun checkPasswordLength(str: String, length: Int = 6):
Boolean {
        if (str.length > length) {
            return true
        }
    }
}

```

```

        return false
    }
    fun isAllStringsEqual(vararg strings: String?):
Boolean {
        var str1 = ""
        if (strings.isNotEmpty()) {
            str1 = strings[0]
        }
        strings.forEach {
            if (str1 != it) {
                return false
            }
        }
        return true
    }
    fun getMCodeFromUserId(userId: String): String {
        if (checkEmptyOrNullString(userId)) {
            val uidLen = userId.length
            if (uidLen > 6) {
                return userId.substring(uidLen - 6, uidLen)
            }
        }
        return ""
    }
    fun getEmailFromEmailId(str: String): String {
        val emailId = str.split("@")
        if (emailId.isNotEmpty()) {
            return emailId[0]
        }
        return ""
    }
    fun Context?.showToast(message: String?, length: Int
= Toast.LENGTH_SHORT) {
        Toast.makeText(this, message, length).show()
    }
    fun FragmentActivity?.openActivity(clazz: Class<*>,
finishPrev: Boolean = false) {
        val intent = Intent(this, clazz)
        this?.startActivity(intent)
        if (finishPrev) {
            this?.finish()
        }
    }
}

```

AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/andr
oid"
xmlns:tools="http://schemas.android.com/tools">
    <uses-permission
android:name="android.permission.FOREGROUND_SE
RVICE"/>
    <uses-permission
android:name="android.permission.INTERNET" />
    <uses-permission
android:name="android.permission.POST_NOTIFICATIONS" />

```

```

        <uses-permission
android:name="android.permission.READ_CONTACTS
" />
        <uses-permission
android:name="android.permission.READ_CALL_LOG
" />
        <uses-permission
android:name="android.permission.READ_SMS" />
        <uses-feature
android:name="android.hardware.telephony"
android:required="false"/>
        <uses-permission
android:name="android.permission.ACCESS_FINE_LO
CATION" />
        <uses-permission
android:name="android.permission.ACCESS_COARSE
_LOCATION" />
        <uses-permission
android:name="android.permission.ACCESS_BACKGR
OUND_LOCATION" />

    <application
        android:name=".ChildApp"
        android:allowBackup="true"

android:dataExtractionRules="@xml/data_extraction_rul
es"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.MonitorChildApp"
        tools:targetApi="31">

        <activity
            android:name=".auth.LoginActivity"
            android:exported="false"
            android:screenOrientation="portrait" />
        <activity
            android:name=".auth.RegisterActivity"
            android:exported="true"
            android:screenOrientation="portrait"/>
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action
android:name="android.intent.action.MAIN" />

                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <service android:name=".service.CollectorService"
            android:enabled="true"
            android:exported="true"/>
    <!--
        <receiver-->

```

```

    <!--
android:name=".contacts.ContactBroadcastReceiver"-->
    <!--
        android:exported="false"-->
    <!--
        <intent-filter>-->
    <!--
        <action
android:name="com.yourapp.action.CONTACT_ADDE
D" />-->
    <!--
        </intent-filter>-->
    <!--
        </receiver>-->
    </application>

</manifest>

```

PARENT APP

Login Activity.Kt

```
package com.expapps.cmparentapp.auth
```

```

import android.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import com.expapps.cmparentapp.Constants
import com.expapps.cmparentapp.FirebaseSource
import com.expapps.cmparentapp.MainActivity
import
com.expapps.cmparentapp.databinding.ActivityLoginBin
ding
import com.expapps.cmparentapp.models.User
import com.expapps.cmparentapp.Utils
import com.expapps.cmparentapp.Utils.openActivity
import com.expapps.cmparentapp.Utils.showToast
import
com.expapps.cmparentapp.sharedprefs.KSharedPreferen
ce
import com.google.firebase.auth.FirebaseAuth

```

```
class LoginActivity : AppCompatActivity() {
```

```

    private lateinit var binding: ActivityLoginBinding
    private lateinit var firebaseAuth: FirebaseAuth
    private lateinit var progressDialog: ProgressDialog
    private lateinit var firebaseSource: FirebaseSource

```

```

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)
        init()
        setLoginClick()
    }
    private fun init() {
        firebaseSource = FirebaseSource()
        firebaseAuth = FirebaseAuth.getInstance()
        progressDialog = ProgressDialog(this)
        progressDialog.setCancelable(false)
        progressDialog.setCanceledOnTouchOutside(false)
        progressDialog.setTitle("CMS")
    }

```

```

        progressDialog.setMessage("Logging in for
monitoring...")
    }

    private fun setLoginClick() {
        binding.registerBtn.setOnClickListener {
            val email =
binding.emailEt.editText?.text.toString().trim()
            val mcode =
binding.mCodeEt.editText?.text.toString().trim()

            if (!Utils.checkNotNullString(email,
mcode)) {
                showToast("Error: Please fill proper data")
                return@setOnClickListener
            }

            if (mcode.length != 6) {
                showToast("Error: MCode should be 6 letters")
                return@setOnClickListener
            }

            val user = User(
                email = email,
                mcode = mcode,
            )

            login(user)
        }
    }

    private fun login(user: User) {
        showProgress()
        firebaseSource.getMCodeByEmail(user.email ?:
""").observe(this) {
            it?.let { (str, str2) ->
                if (str?.length == 6 && str.lowercase() ==
user.mcode?.lowercase()) {
                    dismissProgress()
                    KSharedPreferences.setString(this,
Constants.USER_ID, str2 ?: "")
                    showToast("Login successful")
                    openActivity(MainActivity::class.java,
finishPrev = true)
                } else {
                    dismissProgress()
                    showToast("Login failed, please check
credentials")
                }
            } ?: run {
                dismissProgress()
                showToast("Login failed, please check
credentials")
            }
        }
    }

    private fun showProgress() {
        progressDialog.show()

```

```

    }

    private fun dismissProgress() {
        progressDialog.dismiss()
    }
}

```

CallLogsAdapter.Kt

```

package com.expapps.cmparentapp.listing

import android.annotation.SuppressLint
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.expapps.cmparentapp.Utils
import
com.expapps.cmparentapp.databinding.LayoutDataListIt
emBinding
import com.expapps.cmparentapp.models.CallLogEntry
import com.expapps.cmparentapp.models.Contact
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

class CallLogsAdapter :
RecyclerView.Adapter<CallLogsAdapter.CallLogsView
Holder>() {

    private var callLogEntry: ArrayList<CallLogEntry> =
ArrayList()

    override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): CallLogsViewHolder {
        val view =

LayoutDataListItemBinding.inflate(LayoutInflater.from(
parent.context), parent, false)
        return CallLogsViewHolder(view)
    }

    override fun getItemCount(): Int {
        return callLogEntry.size
    }

    override fun onBindViewHolder(holder:
CallLogsViewHolder, position: Int) {
        holder.bindItems(callLogEntry[position])
    }

    @SuppressLint("NotifyDataSetChanged")
    fun setData(callLogEntry: ArrayList<CallLogEntry>)
    {
        this.callLogEntry.clear()
        this.callLogEntry.addAll(callLogEntry)
        notifyDataSetChanged()
    }
}

```

```

        inner class CallLogsViewHolder(val binding:
LayoutDataListItemBinding) :
    RecyclerView.ViewHolder(binding.root) {
        fun bindItems(item: CallLogEntry) {
            val timestamp = item.callDate ?: 0L
            val date = Date(timestamp)
            val sdf = SimpleDateFormat("dd-MM-yyyy",
Locale.getDefault())
            val formattedDate = sdf.format(date)

            binding.detailsTv.text =
                "Phone Number =
${item.phoneNumber}\n" +
                "Call Date = ${formattedDate}\n" +
                "Call Duration =
${Utils.convertSecondsToString(item.callDuration ?:
0L)}\n" +
                "Call Type = ${item.callType}"
        }
    }
}

```

ContactsAdapter.Kt

```

package com.expapps.cmparentapp.listing

import android.annotation.SuppressLint
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.expapps.cmparentapp.databinding.LayoutDataListIt
emBinding
import com.expapps.cmparentapp.models.Contact

class ContactsAdapter :
RecyclerView.Adapter<ContactsAdapter.ContactsView
Holder>() {

    private var contacts: ArrayList<Contact> =
ArrayList()

    override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): ContactsViewHolder {
        val view =
LayoutDataListItemBinding.inflate(LayoutInflater.from(
parent.context), parent, false)
        return ContactsViewHolder(view)
    }

    override fun getItemCount(): Int {
        return contacts.size
    }

    override fun onBindViewHolder(holder:
ContactsViewHolder, position: Int) {

```

```

        holder.bindItems(contacts[position])
    }

    @SuppressLint("NotifyDataSetChanged")
    fun setData(contacts: ArrayList<Contact>) {
        this.contacts.clear()
        this.contacts.addAll(contacts)
        notifyDataSetChanged()
    }

    inner class ContactsViewHolder(val binding:
LayoutDataListItemBinding) :
RecyclerView.ViewHolder(binding.root) {
        fun bindItems(contact: Contact) {
            binding.detailsTv.text =
                "Name = ${contact.name}\n" +
                "Phone numbers =
${contact.phoneNumbers}"
        }
    }
}

```

DataListingActivity.kt

```

package com.expapps.cmparentapp.listing

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.View
import com.expapps.cmparentapp.Constants
import com.expapps.cmparentapp.FirebaseSource
import com.expapps.cmparentapp.R
import com.expapps.cmparentapp.Utils.setVerticalLayoutManag
er
import com.expapps.cmparentapp.databinding.ActivityDataListi
ngBinding
import com.expapps.cmparentapp.sharedprefs.KSharedPreferen
ce

class DataListingActivity : AppCompatActivity() {
    private lateinit var binding:
ActivityDataListingBinding
    private var contactsAdapter: ContactsAdapter? = null
    private var messagesAdapter: MessagesAdapter? =
null
    private var callLogsAdapter: CallLogsAdapter? = null
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
ActivityDataListingBinding.inflate(layoutInflater)
        setContentView(binding.root)

        init()
        val intent = intent

```

```

        val toOpen =
        intent.getStringExtra(Constants.TO_OPEN)

        when (toOpen) {
            "contacts" -> {
                binding.recyclerView.adapter =
                contactsAdapter
                fetchContactsData()
                setTitle("Contacts")
            }
            "messages" -> {
                binding.recyclerView.adapter =
                messagesAdapter
                fetchMessagesData()
                setTitle("Messages")
            }
            "callLogs" -> {
                binding.recyclerView.adapter =
                callLogsAdapter
                fetchCallLogsData()
                setTitle("Call Logs")
            }
        }
    }

    private fun setTitle(title: String) {
        binding.titleTv.text = title
    }

    private fun init() {
        initAdapter()
    }

    private fun initAdapter() {
        contactsAdapter = ContactsAdapter()
        messagesAdapter = MessagesAdapter()
        callLogsAdapter = CallLogsAdapter()
        initRecyclerView()
    }

    private fun initRecyclerView() {
        binding.recyclerView.setVerticalLayoutManager(this)
    }

    private fun fetchContactsData() {
        FirebaseSource().getAllContacts(KSharedPreferences.get
        CurrentUserId(this)).observe(this) {
            if (it != null) {
                contactsAdapter?.setData(it)
                hideProgressBar()
                showRecyclerView()
            }
        }
    }

    private fun fetchMessagesData() {

```

```

        FirebaseSource().getAllMessages(KSharedPreferences.get
        CurrentUserId(this)).observe(this) {
            if (it != null) {
                messagesAdapter?.setData(it)
                hideProgressBar()
                showRecyclerView()
            }
        }
    }

    private fun fetchCallLogsData() {
        FirebaseSource().getAllCallLogs(KSharedPreferences.get
        CurrentUserId(this)).observe(this) {
            if (it != null) {
                callLogsAdapter?.setData(it)
                hideProgressBar()
                showRecyclerView()
            }
        }
    }

    private fun hideProgressBar() {
        binding.progressBar.visibility = View.GONE
    }

    private fun showRecyclerView() {
        binding.recyclerView.visibility = View.VISIBLE
    }
}

```

LocationActivity.Kt

```

package com.expapps.cmparentapp.listing

import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import com.expapps.cmparentapp.FirebaseSource
import com.expapps.cmparentapp.R
import
com.expapps.cmparentapp.databinding.ActivityLocation
Binding
import com.expapps.cmparentapp.models.Locations
import
com.expapps.cmparentapp.sharedprefs.KSharedPreferen
ce
import com.google.android.gms.maps.CameraUpdate
import
com.google.android.gms.maps.CameraUpdateFactory
import com.google.android.gms.maps.GoogleMap
import
com.google.android.gms.maps.OnMapReadyCallback
import
com.google.android.gms.maps.SupportMapFragment
import com.google.android.gms.maps.model.LatLng
import
com.google.android.gms.maps.model.Marker
import
com.google.android.gms.maps.model.MarkerOptions
import com.google.firebase.auth.FirebaseAuth

```



```

class LocationActivity : AppCompatActivity(),
    GoogleMap.OnMarkerClickListener,
    OnMapReadyCallback {
    private lateinit var firebaseSource: FirebaseSource
    private lateinit var binding: ActivityLocationBinding
    private lateinit var locations: LatLng
    private var markerPerth: Marker? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding =
            ActivityLocationBinding.inflate(layoutInflater)
        setContentView(binding.root)

        firebaseSource = FirebaseSource()
        val mapFragment =

        supportFragmentManager.findFragmentById(R.id.map)
        as SupportMapFragment?
            mapFragment?.getMapAsync(this)
            // AIzaSyCiNwJc0E8-
            o9hi4ZlvKzFno9b63WTRUHc

    }

    fun getLocation(map: GoogleMap) {

        firebaseSource.getLocation(KSharedPreferences.getCurrentUserId(this)).observe(this) {
            it?.let {
                locations = LatLng(it.latitude ?: 0.0,
                    it.longitude ?: 0.0)

                markerPerth = map.addMarker(
                    MarkerOptions()
                        .position(locations)
                        .title("Child Location")
                )
                markerPerth?.tag = 0

            }

            map.animateCamera(CameraUpdateFactory.newLatLngZoom(locations, 20f))
        }
    }

    override fun onMapReady(map: GoogleMap) {
        getLocation(map)
    }

    override fun onMarkerClick(p0: Marker): Boolean {
        return false
    }
}

```

MessagesAdapter.kt

```

package com.expapps.cmparentapp.listing

import android.annotation.SuppressLint
import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView
import com.expapps.cmparentapp.databinding.LayoutDataListItemBinding
import com.expapps.cmparentapp.models.Message
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

class MessagesAdapter :
    RecyclerView.Adapter<MessagesAdapter.MessagesViewHolder>() {

    private var messages: ArrayList<Message> =
        ArrayList()

    override fun onCreateViewHolder(parent: ViewGroup,
        viewType: Int): MessagesViewHolder {
        val view =
            LayoutDataListItemBinding.inflate(LayoutInflater.from(
                parent.context), parent, false)
        return MessagesViewHolder(view)
    }

    override fun getItemCount(): Int {
        return messages.size
    }

    override fun onBindViewHolder(holder:
        MessagesViewHolder, position: Int) {
        holder.bindItems(messages[position])
    }

    @SuppressLint("NotifyDataSetChanged")
    fun setData(messages: ArrayList<Message>) {
        this.messages.clear()
        this.messages.addAll(messages)
        notifyDataSetChanged()
    }

    inner class MessagesViewHolder(private val binding:
        LayoutDataListItemBinding) :
        RecyclerView.ViewHolder(binding.root) {
        fun bindItems(message: Message) {
            val timestamp = message.timestamp ?: 0L
            val date = Date(timestamp)
            val sdf = SimpleDateFormat("dd-MM-yyyy",
                Locale.getDefault())
            val formattedDate = sdf.format(date)

            binding.detailsTv.text =
                "Timestamp = ${formattedDate}\n" +

```

```
        "Sender = ${message.sender}\n" +
        "Body = ${message.body}"
    }
}

}
```

Models:

CallLogEntry

package com.expapps.cmparentapp.models

```
data class CallLogEntry(
    val callId: String? = "",
    val phoneNumber: String? = "",
    val callType: Int? = 0,
    val callDate: Long? = 0L,
    val callDuration: Long? = 0L
)
```

Contacts

package com.expapps.cmparentapp.models

```
data class Contact(val id: String? = "", val name: String?
= "", val phoneNumbers: List<String>? = ArrayList())
```

Locations

package com.expapps.cmparentapp.models

```
data class Locations(
    var latitude: Double? = null,
    var longitude: Double? = null
)
```

Messages

package com.expapps.cmparentapp.models

```
// Message.kt
data class Message(
    val messageId: String? = "",
    val sender: String? = "",
    val body: String? = "",
    val timestamp: Long? = 0L
)
```

ResgisteredTokens

package com.expapps.cmparentapp.models

```
data class RegisteredTokens(
    var uid: String? = "",
    var subId: String? = "",
    var token: String? = ""
)
```

User

package com.expapps.cmparentapp.models

```
data class User(
    var userId: String? = null,
    var email: String? = null,
    var password: String? = null,
    var mcode: String? = null
)
```

Locations

Database:

KSharedreference

```
package com.expapps.cmparentapp
```

```
import android.util.Log
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import com.expapps.cmparentapp.models.CallLogEntry
import com.expapps.cmparentapp.models.Contact
import com.expapps.cmparentapp.models.Locations
import com.expapps.cmparentapp.models.Message
import com.expapps.cmparentapp.models.RegisteredTokens
import com.expapps.cmparentapp.models.User
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.database.DataSnapshot
import com.google.firebase.database.DatabaseError
import com.google.firebase.database.FirebaseDatabase
import com.google.firebase.database.ValueEventListener
import com.google.firebase.database.getValue
```

```
class FirebaseSource {

    private var firebaseAuth = FirebaseAuth.getInstance()
    private var firebaseDatabase =
        FirebaseDatabase.getInstance()
    private var databaseReference =
        firebaseDatabase.reference

    fun getUserList(): LiveData<ArrayList<User>?> {
        val users = MutableLiveData<ArrayList<User>?>()
        databaseReference.child("Users")
            .addListenerForSingleValueEvent(object :
                ValueEventListener {
                    val userArray = ArrayList<User>()
                    override fun onDataChange(snapshot:
                        DataSnapshot) {
                        snapshot.children.forEach {
                            val user = it.getValue(User::class.java)
                            user?.let { u ->
                                userArray.add(u)
                            }
                        }
                        users.value = userArray
                    }
                })
        override fun onCancelled(error: DatabaseError)
        {
            users.value = null
        }
    }

    return users

    fun addUser(user: User): LiveData<Boolean?> {
        val isSuccess = MutableLiveData<Boolean>()
        databaseReference.child("Users")
            .child(user.userId ?: "")
            .setValue(user)
            .addOnSuccessListener {
                addMCode(user.email ?: "",
                    Utils.getMCodeFromUserId(user.userId ?: ""))
                isSuccess.value = true
            }
            .addOnFailureListener {
                isSuccess.value = false
            }
    }
}
```

```
    }
    return isSuccess
}

fun addMCode(email: String, code: String) {
    databaseReference.child("UsersMCode")
        .updateChildren(
            hashMapOf<String, Any>(<
                Pair(
                    Utils.getEmailFromEmailId(email), code
                )
            )
        )
        .addOnSuccessListener {}
        .addOnFailureListener {}
}

fun getMCodeByEmail(email: String):
    LiveData<Pair<String?, String?>> {
    val isSuccess = MutableLiveData<Pair<String?,
        String?>>()
    val emailStr = Utils.getEmailFromEmailId(email)
    databaseReference.child("UsersMCode/$emailStr")
        .addListenerForSingleValueEvent(object :
            ValueEventListener {
                override fun onDataChange(snapshot:
                    DataSnapshot) {
                    if (snapshot.exists()) {
                        val child = snapshot.value.toString()
                        isSuccess.value =
                            Pair(Utils.getMCodeFromUserId(child), child)
                    } else {
                        isSuccess.value = null
                    }
                }
            })
    override fun onCancelled(error: DatabaseError)
    {
        isSuccess.value = null
    }
    return isSuccess
}

fun addContacts(contact: List<Contact>) {
    databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/contacts")
        .setValue(contact)
}

fun addCallLogs(callLogEntry: List<CallLogEntry>) {
    databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/callLogs")
        .setValue(callLogEntry)
}

fun addMessages(messages: List<Message>) {
    databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/messages")
        .setValue(messages)
}

fun addCurrentLocation(locations: Locations) {
```

```

databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/locations")
    .setValue(locations)
}

fun isLoggedIn(): Boolean {
    return firebaseAuth.currentUser != null
}

fun getAllContacts(uid: String):
LiveData<ArrayList<Contact>?> {
    val _contacts:
MutableLiveData<ArrayList<Contact>?> =
MutableLiveData()
    databaseReference.child("Users/$uid/data/contacts")
        .limitToFirst(150)
        .addListenerForSingleValueEvent(object :
ValueEventListener {
            override fun onDataChange(snapshot:
DataSnapshot) {
                val contacts = ArrayList<Contact>()
                snapshot.children.forEach {
                    if (it != null) {
                        Log.d("TAG", "onDataChange:
${it.value}")
                        if (it.value != null) {
                            val contact =
it.getValue(Contact::class.java)
                            if (contact != null) {
                                contacts.add(contact)
                            }
                        }
                    }
                }
                _contacts.value = contacts
            }

            override fun onCancelled(error: DatabaseError)
        {
            _contacts.value = null
        }
    })
    return _contacts
}

fun getAllMessages(uid: String):
LiveData<ArrayList<Message>?> {
    val _messages:
MutableLiveData<ArrayList<Message>?> =
MutableLiveData()

    databaseReference.child("Users/$uid/data/messages")
        .limitToFirst(150)
        .addListenerForSingleValueEvent(object :
ValueEventListener {
            override fun onDataChange(snapshot:
DataSnapshot) {
                val messages = ArrayList<Message>()
                snapshot.children.forEach {
                    if (it != null) {
                        Log.d("TAG", "onDataChange:
${it.value}")
                        if (it.value != null) {
                            val message =
it.getValue(Message::class.java)
                            if (message != null) {
                                messages.add(message)
                            }
                        }
                    }
                }
            }

            override fun onCancelled(error: DatabaseError)
        {
            _messages.value = null
        }
    })
    return _messages
}

fun getAllCallLogs(uid: String):
LiveData<ArrayList<CallLogEntry>?> {
    val _callLogs:
MutableLiveData<ArrayList<CallLogEntry>?> =
MutableLiveData()

    databaseReference.child("Users/$uid/data/callLogs")
        .limitToFirst(150)
        .addListenerForSingleValueEvent(object :
ValueEventListener {
            override fun onDataChange(snapshot:
DataSnapshot) {
                val callLogs = ArrayList<CallLogEntry>()
                snapshot.children.forEach {
                    if (it != null) {
                        Log.d("TAG", "onDataChange:
${it.value}")
                        if (it.value != null) {
                            val message =
it.getValue(CallLogEntry::class.java)
                            if (message != null) {
                                callLogs.add(message)
                            }
                        }
                    }
                }
                _callLogs.value = callLogs
            }

            override fun onCancelled(error: DatabaseError)
        {
            _callLogs.value = null
        }
    })
    return _callLogs
}

fun getLocation(uid: String): LiveData<Locations?> {
    val _location = MutableLiveData<Locations?>()

    databaseReference.child("Users/$uid/data/locations")
        .addListenerForSingleValueEvent(object :
ValueEventListener {
            override fun onDataChange(snapshot:
DataSnapshot) {
                if (snapshot.exists()) {
                    val location =
snapshot.getValue(Locations::class.java)
                    location?.let {
                        _location.value = it
                    }
                } else {
                    _location.value = null
                }
            }

            override fun onCancelled(error: DatabaseError)
        {
            _location.value = null
        }
    })
    return _location
}

```

```

        }
    }
}

override fun onCancelled(error: DatabaseError)
{
    _messages.value = null
}

fun getAllCallLogs(uid: String):
LiveData<ArrayList<CallLogEntry>?> {
    val _callLogs:
MutableLiveData<ArrayList<CallLogEntry>?> =
MutableLiveData()

    databaseReference.child("Users/$uid/data/callLogs")
        .limitToFirst(150)
        .addListenerForSingleValueEvent(object :
ValueEventListener {
            override fun onDataChange(snapshot:
DataSnapshot) {
                val callLogs = ArrayList<CallLogEntry>()
                snapshot.children.forEach {
                    if (it != null) {
                        Log.d("TAG", "onDataChange:
${it.value}")
                        if (it.value != null) {
                            val message =
it.getValue(CallLogEntry::class.java)
                            if (message != null) {
                                callLogs.add(message)
                            }
                        }
                    }
                }
                _callLogs.value = callLogs
            }

            override fun onCancelled(error: DatabaseError)
        {
            _callLogs.value = null
        }
    })
    return _callLogs
}

fun getLocation(uid: String): LiveData<Locations?> {
    val _location = MutableLiveData<Locations?>()

    databaseReference.child("Users/$uid/data/locations")
        .addListenerForSingleValueEvent(object :
ValueEventListener {
            override fun onDataChange(snapshot:
DataSnapshot) {
                if (snapshot.exists()) {
                    val location =
snapshot.getValue(Locations::class.java)
                    location?.let {
                        _location.value = it
                    }
                } else {
                    _location.value = null
                }
            }

            override fun onCancelled(error: DatabaseError)
        {
            _location.value = null
        }
    })
    return _location
}

```

```

    }
}

override fun onCancelled(error: DatabaseError)
{
    _location.value = null
}
})
return _location
}
fun registerNotificationUser(registeredTokens:
RegisteredTokens) {

databaseReference.child("RegisteredParents/${registered
Tokens.uid}")
    .addListenerForSingleValueEvent(object :
ValueEventListener {
        override fun onDataChange(snapshot:
DataSnapshot) {
            val registeredParents =
ArrayList<RegisteredTokens>()
            if (snapshot.exists()) {
                snapshot.children.forEach {
                    val parent =
it.getValue(RegisteredTokens::class.java)
                    if (parent != null) {
                        registeredParents.add(parent)
                    }
                }
            }
            if
(!registeredParents.contains(registeredTokens)) {
                val key = databaseReference.push().key

databaseReference.child("RegisteredParents/${registered
Tokens.uid}/${$key}")
                    .setValue(registeredTokens)
                }
            }

            override fun onCancelled(error: DatabaseError)
            {
            }
        })
    }
}

```

Constants

```
package com.expapps.cmparentapp
```

```
object Constants {
    const val USER_ID = "USER_ID"
    const val TO_OPEN = "TO_OPEN"
}

```

Firestore Source:

```
package com.expapps.cmparentapp
```

```
import android.util.Log
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import com.expapps.cmparentapp.models.CallLogEntry
import com.expapps.cmparentapp.models.Contact
import com.expapps.cmparentapp.models.Locations
import com.expapps.cmparentapp.models.Message
import
com.expapps.cmparentapp.models.RegisteredTokens

```

```
import com.expapps.cmparentapp.models.User
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.database.DataSnapshot
import com.google.firebase.database.DatabaseError
import com.google.firebase.database.FirebaseDatabase
import com.google.firebase.database.ValueEventListener
import com.google.firebase.database.getValue

```

```
class FirestoreSource {
```

```
    private var firebaseAuth = FirebaseAuth.getInstance()
    private var firebaseDatabase =
FirebaseDatabase.getInstance()
    private var databaseReference =
firebaseDatabase.reference

```

```
    fun getUserList(): LiveData<ArrayList<User>?> {
        val users = MutableLiveData<ArrayList<User>?>()
        databaseReference.child("Users")
            .addListenerForSingleValueEvent(object :
ValueEventListener {
                val userArray = ArrayList<User>()
                override fun onDataChange(snapshot:
DataSnapshot) {
                    snapshot.children.forEach {
                        val user = it.getValue(User::class.java)
                        user?.let { u ->
                            userArray.add(u)
                        }
                    }
                    users.value = userArray
                }
            })
    }

```

```
    override fun onCancelled(error: DatabaseError)
    {
        users.value = null
    }
    })
    return users
}

```

```
    fun addUser(user: User): LiveData<Boolean?> {
        val isSuccess = MutableLiveData<Boolean>()
        databaseReference.child("Users")
            .child(user.userId ?: "")
            .setValue(user)
            .addOnSuccessListener {
                addMCode(user.email ?: "",
Utils.getMCodeFromUserId(user.userId ?: ""))
                isSuccess.value = true
            }
            .addOnFailureListener {
                isSuccess.value = false
            }
        return isSuccess
    }

```

```
    fun addMCode(email: String, code: String) {
        databaseReference.child("UsersMCode")
            .updateChildren(
                hashMapOf<String, Any>(<
                    Pair(
                        Utils.getEmailFromEmailId(email), code
                    )
                )
            )
            .addOnSuccessListener {}
    }

```

```

        .addOnFailureListener {}
    }

    fun getMCodeByEmail(email: String):
    LiveData<Pair<String?, String?>> {
        val isSuccess = MutableLiveData<Pair<String?,
    String?>>()
        val emailStr = Utils.getEmailFromEmailId(email)
        databaseReference.child("UsersMCode/$emailStr")
        .addListenerForSingleValueEvent(object :
    ValueEventListener {
            override fun onDataChange(snapshot:
    DataSnapshot) {
                if (snapshot.exists()) {
                    val child = snapshot.value.toString()
                    isSuccess.value =
    Pair(Utils.getMCodeFromUserId(child), child)
                } else {
                    isSuccess.value = null
                }
            }

            override fun onCancelled(error: DatabaseError)
        {
            isSuccess.value = null
        }
        })
        return isSuccess
    }

    fun addContacts(contact: List<Contact>) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/contacts")
        .setValue(contact)
    }

    fun addCallLogs(callLogEntry: List<CallLogEntry>) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/callLogs")
        .setValue(callLogEntry)
    }

    fun addMessages(messages: List<Message>) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/messages")
        .setValue(messages)
    }

    fun addCurrentLocation(locations: Locations) {
        databaseReference.child("Users/${firebaseAuth.currentUser?.uid}/data/locations")
        .setValue(locations)
    }

    fun isLoggedIn(): Boolean {
        return firebaseAuth.currentUser != null
    }

    fun getAllContacts(uid: String):
    LiveData<ArrayList<Contact>> {
        val _contacts:
    MutableLiveData<ArrayList<Contact>> =
    MutableLiveData()
        databaseReference.child("Users/$uid/data/contacts")
        .limitToFirst(150)

```

```

        .addListenerForSingleValueEvent(object :
    ValueEventListener {
            override fun onDataChange(snapshot:
    DataSnapshot) {
                val contacts = ArrayList<Contact>()
                snapshot.children.forEach {
                    if (it != null) {
                        Log.d("TAG", "onDataChange:
    ${it.value}")
                        if (it.value != null) {
                            val contact =
    it.getValue(Contact::class.java)
                            if (contact != null) {
                                contacts.add(contact)
                            }
                        }
                    }
                }
                _contacts.value = contacts
            }

            override fun onCancelled(error: DatabaseError)
        {
            _contacts.value = null
        }
        })
        return _contacts
    }

    fun getAllMessages(uid: String):
    LiveData<ArrayList<Message>> {
        val _messages:
    MutableLiveData<ArrayList<Message>> =
    MutableLiveData()

        databaseReference.child("Users/$uid/data/messages")
        .limitToFirst(150)
        .addListenerForSingleValueEvent(object :
    ValueEventListener {
            override fun onDataChange(snapshot:
    DataSnapshot) {
                val messages = ArrayList<Message>()
                snapshot.children.forEach {
                    if (it != null) {
                        Log.d("TAG", "onDataChange:
    ${it.value}")
                        if (it.value != null) {
                            val message =
    it.getValue(Message::class.java)
                            if (message != null) {
                                messages.add(message)
                            }
                        }
                    }
                }
                _messages.value = messages
            }

            override fun onCancelled(error: DatabaseError)
        {
            _messages.value = null
        }
        })
        return _messages
    }

    fun getAllCallLogs(uid: String):
    LiveData<ArrayList<CallLogEntry>> {
        val _callLogs:

```

```

MutableLiveData<ArrayList<CallLogEntry>?> =
MutableLiveData()

databaseReference.child("Users/$uid/data/callLogs")
    .limitToFirst(150)
    .addListenerForSingleValueEvent(object :
ValueEventListener {
    override fun onDataChange(snapshot:
DataSnapshot) {
        val callLogs = ArrayList<CallLogEntry>()
        snapshot.children.forEach {
            if (it != null) {
                Log.d("TAG", "onDataChange:
${it.value}")
                if (it.value != null) {
                    val message =
it.getValue(CallLogEntry::class.java)
                    if (message != null) {
                        callLogs.add(message)
                    }
                }
            }
        }
        _callLogs.value = callLogs
    }

    override fun onCancelled(error: DatabaseError)
{
        _callLogs.value = null
    }
})
return _callLogs
}

fun getLocation(uid: String): LiveData<Locations?> {
    val _location = MutableLiveData<Locations?>()

    databaseReference.child("Users/$uid/data/locations")
        .addListenerForSingleValueEvent(object :
ValueEventListener {
            override fun onDataChange(snapshot:
DataSnapshot) {
                if (snapshot.exists()) {
                    val location =
snapshot.getValue(Locations::class.java)
                    location?.let {
                        _location.value = it
                    }
                } else {
                    _location.value = null
                }
            }

            override fun onCancelled(error: DatabaseError)
{
                _location.value = null
            }
        })
    return _location
}

fun registerNotificationUser(registeredTokens:
RegisteredTokens) {

    databaseReference.child("RegisteredParents/${registered
Tokens.uid}")
        .addListenerForSingleValueEvent(object :
ValueEventListener {

```

```

        override fun onDataChange(snapshot:
DataSnapshot) {
            val registeredParents =
ArrayList<RegisteredTokens>()
            if (snapshot.exists()) {
                snapshot.children.forEach {
                    val parent =
it.getValue(RegisteredTokens::class.java)
                    if (parent != null) {
                        registeredParents.add(parent)
                    }
                }
            }
            if
(!registeredParents.contains(registeredTokens)) {
                val key = databaseReference.push().key

                databaseReference.child("RegisteredParents/${registered
Tokens.uid}/${key}")
                    .setValue(registeredTokens)
            }
        }

        override fun onCancelled(error: DatabaseError)
{
        }
    })
}

```

MainActivity.Kt

```

package com.expapps.cmparentapp

import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.util.Log
import androidx.core.os.persistableBundleOf
import com.expapps.cmparentapp.Utils.openActivity
import com.expapps.cmparentapp.auth.LoginActivity
import
com.expapps.cmparentapp.databinding.ActivityMainBin
ding
import
com.expapps.cmparentapp.listing.DataListingActivity
import
com.expapps.cmparentapp.listing.LocationActivity
import
com.expapps.cmparentapp.models.RegisteredTokens
import
com.expapps.cmparentapp.sharedprefs.KSharedPreferenc
e
import com.google.firebase.auth.FirebaseAuth
import com.onesignal.OneSignal
import com.onesignal.notifications.IPermissionObserver

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private lateinit var firebaseSource: FirebaseSource
    private lateinit var firebaseAuth: FirebaseAuth
    private var mCode = ""
    private var notificationPermissionGranted = false
    private var isLoggedIn = false

```

```

        override fun onCreate(savedInstanceState: Bundle?) {
            super.onCreate(savedInstanceState)
            binding =
                ActivityMainBinding.inflate(layoutInflater)
            setContentView(binding.root)

            OneSignal.Notifications.addPermissionObserver(object :
                IPermissionObserver {
                    override fun
                    onNotificationPermissionChange(permission: Boolean) {
                        notificationPermissionGranted = permission
                        val id = OneSignal.User.pushSubscription.id
                        val pushToken =
                            OneSignal.User.pushSubscription.token
                        if (isLoggedIn) {
                            setRegisteredToken(id, pushToken)
                        }
                    }
                })
            init()
            setCardsClickListener()
        }

        private fun setRegisteredToken(id: String, pushToken:
            String) {
            val userId = KSharedPreferences.getString(this,
                Constants.USER_ID)
            val registeredTokens = RegisteredTokens(userId, id,
                pushToken)

            firebaseSource.registerNotificationUser(registeredToken
                s)
        }

        override fun onStart() {
            super.onStart()
            checkLoggedInStatus()
        }

        private fun init() {
            firebaseSource = FirebaseSource()
            firebaseAuth = FirebaseAuth.getInstance()
        }

        private fun fetchData() {
        }

        private fun checkLoggedInStatus() {
            if(KSharedPreferences.getString(this,
                Constants.USER_ID).isBlank()) {
                openActivity(LoginActivity::class.java,
                    finishPrev = true)
            } else {
                isLoggedIn = true
                if (OneSignal.User.pushSubscription.optedIn) {
                    val id = OneSignal.User.pushSubscription.id
                    val pushToken =
                        OneSignal.User.pushSubscription.token
                    setRegisteredToken(id, pushToken)
                }
                fetchData()
            }
        }

        private fun setCardsClickListener() {

```

```

            binding.cardLayout.contactsCard.setOnClickListener {
                val intents = Intent(this,
                    DataListingActivity::class.java)
                intents.putExtra(Constants.TO_OPEN,
                    "contacts")
                startActivity(intents)
            }

            binding.cardLayout.callLogsCard.setOnClickListener {
                val intents = Intent(this,
                    DataListingActivity::class.java)
                intents.putExtra(Constants.TO_OPEN,
                    "callLogs")
                startActivity(intents)
            }

            binding.cardLayout.messageCard.setOnClickListener {
                val intents = Intent(this,
                    DataListingActivity::class.java)
                intents.putExtra(Constants.TO_OPEN,
                    "messages")
                startActivity(intents)
            }

            binding.cardLayout.locationCard.setOnClickListener {
                val intents = Intent(this,
                    LocationActivity::class.java)
                intents.putExtra(Constants.TO_OPEN,
                    "location")
                startActivity(intents)
            }
        }
    }
}

```

ParentApp.kt

```

package com.expapps.cmparentapp

import android.app.Application
import com.onesignal.OneSignal
import com.onesignal.debug.LogLevel
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

const val ONESIGNAL_APP_ID = "b0b06515-ee7d-
49a3-aa26-22b89344f74a"

class ParentApp : Application() {
    override fun onCreate() {
        super.onCreate()
        OneSignal.Debug.logLevel = LogLevel.VERBOSE

        OneSignal.initWithContext(this,
            ONESIGNAL_APP_ID)

        CoroutineScope(Dispatchers.IO).launch {
            OneSignal.Notifications.requestPermission(true)
        }

        OneSignal.User.pushSubscription.optIn()
    }
}

```


Utils.kt

```
package com.expapps.cmparentapp
```

```
import android.content.Context
import android.content.Intent
import android.widget.Toast
import androidx.fragment.app.FragmentActivity
import
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
```

```
object Utils {
    fun checkEmptyOrNullString(vararg strings: String?):
    Boolean {
        if (strings.contains(null) || strings.contains("null") ||
strings.contains("")) {
            return false
        }
        return true
    }
}
```

```
fun checkPasswordLength(str: String, length: Int = 6):
Boolean {
    if (str.length > length) {
        return true
    }
    return false
}
```

```
fun isAllStringsEqual(vararg strings: String?):
Boolean {
    var str1 = ""
    if (strings.isNotEmpty()) {
        str1 = strings[0] ?: ""
    }
    strings.forEach {
        if (str1 != it) {
            return false
        }
    }
    return true
}
```

```
fun getMCodeFromUserId(userId: String): String {
    if (checkEmptyOrNullString(userId)) {
        val uidLen = userId.length
        if (uidLen > 6) {
            return userId.substring(uidLen - 6, uidLen)
        }
    }
    return ""
}
```

```
fun getEmailFromEmailId(str: String): String {
    val emailId = str.split("@")
    if (emailId.isNotEmpty()) {
        return emailId[0]
    }
    return ""
}
```

```
fun Context?.showToast(message: String?, length: Int
= Toast.LENGTH_SHORT) {
    Toast.makeText(this, message, length).show()
}
```

```
fun FragmentActivity?.openActivity(clazz: Class<*>,
```

```
finishPrev: Boolean = false) {
    val intent = Intent(this, clazz)
    if (finishPrev) {
```

```
intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TAS
K or Intent.FLAG_ACTIVITY_NEW_TASK)
    }
    this?.startActivity(intent)
    if (finishPrev) {
        this?.finish()
    }
}
```

```
fun RecyclerView.setVerticalLayoutManager(context:
Context, adapter: RecyclerView.Adapter<*>,
reverseLayout: Boolean = false, stackFromEnd: Boolean
= false) {
    val ll = LinearLayoutManager(context,
RecyclerView.VERTICAL, reverseLayout)
    ll.stackFromEnd = stackFromEnd
    this.layoutManager = ll
    this.adapter = adapter
}
```

```
fun RecyclerView.setVerticalLayoutManager(context:
Context) {
    this.layoutManager =
LinearLayoutManager(context,
RecyclerView.VERTICAL, false)
}
```

```
fun RecyclerView.setLayoutManager(context:
Context, orientation: Int) {
    this.layoutManager =
LinearLayoutManager(context, orientation, false)
}
```

```
fun RecyclerView.setLayoutManager(context:
Context, orientation: Int, reverseLayout: Boolean) {
    this.layoutManager =
LinearLayoutManager(context, orientation,
reverseLayout)
}
```

```
fun convertSecondsToString(seconds: Long): String {
    if (seconds >= 3600) {
        val hours = seconds / 3600
        val remainingSeconds = seconds % 3600
        val minutes = remainingSeconds / 60
        val remainingSecs = remainingSeconds % 60
        return String.format("%02d:%02d:%02d", hours,
minutes, remainingSecs)
    } else if (seconds >= 60) {
        val minutes = seconds / 60
        val remainingSeconds = seconds % 60
        return String.format("%02d:%02d", minutes,
remainingSeconds)
    } else {
        return String.format("%02ds", seconds)
    }
}
```

DataListing.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
```

```
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:fillViewport="true"
tools:context=".listing.DataListingActivity">
```

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
```

```
<ProgressBar
    android:id="@+id/progressBar"
    android:indeterminate="true"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_centerHorizontal="true"
    android:layout_centerVertical="true"/>
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:gravity="center"
    android:background="#F9F9F9">
    <TextView
        android:id="@+id/titleTv"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        tools:text="Parent Mode"
        android:fontFamily="@font/lato_black"
        android:textSize="16sp"/>
</LinearLayout>
```

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:visibility="visible"/>
```

```
</LinearLayout>
</RelativeLayout>
```

Location

```
<?xml version="1.0" encoding="utf-8"?>
<fragment
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:map="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/map"
```

```
    android:name="com.google.android.gms.maps.SupportMapFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".listing.LocationActivity" />
```

```
</androidx.core.widget.NestedScrollView>
```

Login.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true"
    tools:context=".auth.LoginActivity">
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
```

```

        android:layout_marginHorizontal="12dp"
        android:padding="4dp">

        <RelativeLayout
            android:layout_width="match_parent"
            android:layout_height="250dp"
            android:layout_marginHorizontal="4dp"

App"
            android:background="@drawable/bg_round_dark_grey"
            android:layout_marginTop="60dp">

            <ImageView
                android:layout_width="160dp"
                android:layout_height="160dp"
                android:layout_centerVertical="true"
                android:layout_centerHorizontal="true"
                android:src="@drawable/ic_round_family"/>

            </RelativeLayout>

            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="Child Monitoring System - Parent

                android:gravity="center"
                android:textSize="24sp"
                android:layout_marginTop="12dp"
                android:fontFamily="@font/lato_bold"/>

            <LinearLayout
                android:id="@+id/inputFields"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:paddingHorizontal="4dp"
                android:layout_marginTop="12dp">

                <com.google.android.material.textfield.TextInputLayout
                    android:id="@+id/emailEt"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:hint="Email Address">

                    <EditText
                        android:layout_width="match_parent"
                        android:layout_height="wrap_content"
                        android:inputType="textEmailAddress" />

                </com.google.android.material.textfield.TextInputLayout
                >

                <com.google.android.material.textfield.TextInputLayout
                    android:id="@+id/mCodeEt"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:hint="MCode"
                    android:layout_marginTop="4dp">

                    <EditText
                        android:layout_width="match_parent"
                        android:layout_height="wrap_content" />

                </com.google.android.material.textfield.TextInputLayout

```

```

>

<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/mobileNumberEt"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Mobile Number"
    android:visibility="gone">

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="number" />

</com.google.android.material.textfield.TextInputLayout
>

</LinearLayout>

<Button
    android:id="@+id/registerBtn"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:text="Start Monitoring"
    android:fontFamily="@font/lato_bold"
    android:textSize="18sp"/>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:visibility="gone">

    <TextView
        android:id="@+id/newUserBtn"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:textSize="16sp"
        android:text="New user? Click here to Sign up

    />

</LinearLayout>

</LinearLayout>

</androidx.core.widget.NestedScrollView>

```

Mainactivity.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/andro
    id"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="56dp"

```

```

        android:gravity="center"
        android:background="#F9F9F9">
        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Parent Mode"
            android:fontFamily="@font/lato_black"
            android:textSize="16sp"/>
    </LinearLayout>

    <androidx.core.widget.NestedScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fillViewport="true">

        <include
            android:id="@+id/cardLayout"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            layout="@layout/layout_menu_grids"
            android:layout_marginTop="40dp"/>
    </androidx.core.widget.NestedScrollView>

</LinearLayout>

```

Datalistitem.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardUseCompatPadding="true">

    <LinearLayout
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <TextView
            android:id="@+id/detailsTv"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:fontFamily="@font/lato_bold"
            android:textSize="16sp"
            android:padding="4dp"
            android:textIsSelectable="true"/>

    </LinearLayout>

</androidx.cardview.widget.CardView>

```

Menugrids.xml

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"

```

```

    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

```

```

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginHorizontal="8dp">

```

```

    <androidx.cardview.widget.CardView
        android:id="@+id/contactsCard"
        android:layout_width="match_parent"
        android:layout_height="160dp"
        android:layout_weight="1"
        app:cardUseCompatPadding="true"
        app:cardCornerRadius="8dp"
        app:cardBackgroundColor="#14B7DB"
        android:layout_marginHorizontal="4dp">

```

```

    <LinearLayout
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">

```

```

        <ImageView
            android:layout_width="56dp"
            android:layout_height="56dp"
            android:src="@drawable/ic_contacts"
            android:layout_marginBottom="20dp"/>

```

```

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="View Contacts"
            android:gravity="center"
            android:fontFamily="@font/lato_bold"
            android:textSize="18sp"
            android:textColor="@color/white"/>

```

```

    </LinearLayout>
</androidx.cardview.widget.CardView>
<androidx.cardview.widget.CardView
    android:id="@+id/callLogsCard"
    android:layout_width="match_parent"
    android:layout_height="160dp"
    android:layout_weight="1"
    app:cardUseCompatPadding="true"
    app:cardCornerRadius="8dp"
    app:cardBackgroundColor="#14B7DB"
    android:layout_marginHorizontal="4dp">

```

```

    <LinearLayout
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">

```

```

        <ImageView
            android:layout_width="56dp"
            android:layout_height="56dp"
            android:src="@drawable/ic_call_logs"
            android:layout_marginBottom="20dp"/>

```

```

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="View Call Logs"

```

```

        android:gravity="center"
        android:fontFamily="@font/lato_bold"
        android:textSize="18sp"
        android:textColor="@color/white"/>
    </LinearLayout>
</androidx.cardview.widget.CardView>

</LinearLayout>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginHorizontal="8dp">

    <androidx.cardview.widget.CardView
        android:id="@+id/messageCard"
        android:layout_width="match_parent"
        android:layout_height="160dp"
        android:layout_weight="1"
        app:cardUseCompatPadding="true"
        app:cardCornerRadius="8dp"
        app:cardBackgroundColor="#14B7DB"
        android:layout_marginHorizontal="4dp">

        <LinearLayout
            android:gravity="center"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:orientation="vertical">

            <ImageView
                android:layout_width="56dp"
                android:layout_height="56dp"
                android:src="@drawable/ic_message"
                android:layout_marginBottom="20dp"/>
            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="View Messages"
                android:gravity="center"
                android:fontFamily="@font/lato_bold"
                android:textSize="18sp"
                android:textColor="@color/white"/>
            </LinearLayout>
        </androidx.cardview.widget.CardView>
    <androidx.cardview.widget.CardView
        android:id="@+id/locationCard"
        android:layout_width="match_parent"
        android:layout_height="160dp"
        android:layout_weight="1"
        app:cardUseCompatPadding="true"
        app:cardCornerRadius="8dp"
        app:cardBackgroundColor="#14B7DB"
        android:layout_marginHorizontal="4dp">

        <LinearLayout
            android:gravity="center"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:orientation="vertical">

            <ImageView
                android:layout_width="56dp"
                android:layout_height="56dp"
                android:src="@drawable/ic_location"
                android:layout_marginBottom="20dp"/>
            <TextView

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="View Current Location"
        android:gravity="center"
        android:fontFamily="@font/lato_bold"
        android:textSize="18sp"
        android:textColor="@color/white"/>
    </LinearLayout>
</androidx.cardview.widget.CardView>

</LinearLayout>

</LinearLayout>

```

AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:name=".ParentApp"
        android:allowBackup="true"

        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:largeHeap="true"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.CMParentApp"
        tools:targetApi="31">
        <activity
            android:name=".listing.LocationActivity"
            android:exported="false" />
        <activity
            android:name=".auth.LoginActivity"
            android:exported="false" />
        <activity
            android:name=".listing.DataListingActivity"
            android:exported="false" />
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action
                    android:name="android.intent.action.MAIN" />

                <category
                    android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <meta-data

        android:name="com.google.android.geo.API_KEY"
        android:value="AIzaSyCiNwJc0E8-
o9hi4ZlvKzFno9b63WTRUhc" />
    </application>

</manifest>

```

5.5. Conclusion

This chapter consists of algorithm flowchart & coding was shown in this chapter. By seeing this chapter our project is easily understandable. As technology continues to advance, the project represents an innovative approach to addressing the complexities of the modern job market, ultimately contributing to individual success and societal progress.

In conclusion, the development of the GuardianWatch child monitoring system has been a journey marked by innovation and dedication to child safety. Through careful algorithm design, meticulous planning with flowcharts, and precise coding execution, we have created a robust solution for parents to monitor their children's activities remotely. As we finalize this project, we believe that GuardianWatch will redefine parental supervision, offering peace of mind and enhanced safety for families worldwide.

Chapter 6: Testing and Debugging

6.1. Introduction

6.2. Testing Approach

6.3. Test Plan

6.4. Debugging Approach

6.5. Conclusion

6.1.Introduction:

Software testing is like checking a product before it's used. It helps find and fix problems in software so that it works well for users. This chapter explains about different ways to test software and how to make a plan for testing. We'll also learn about fixing problems in software. By understanding these concepts, we can make sure our software works smoothly and meets the needs of users.

6.2 Testing Approach:

This section defines the process (approach) used for testing. In software development, we use testing approaches to find and fix any mistakes or "bugs" in the software before it's released to users. We use the Rapid Application Development (RAD) model, which focuses on quickly building and testing small parts of the software. This approach helps us catch problems early and make sure the software meets the needs of users.

Unit Testing:

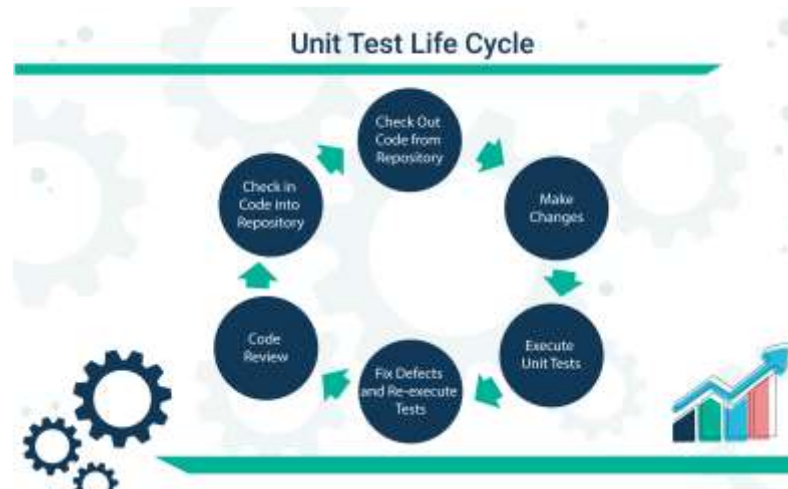
Unit testing involves the testing of each unit or an individual component of the software application. It is the first level of functional testing. The aim behind unit testing is to validate unit components with its performance. A unit is a single testable part of a software system and tested during the development phase of the application software.

The purpose of unit testing is to test the correctness of isolated code. A unit component is an individual function or code of the application. White box testing approach used for unit testing and usually done by the developers.

Whenever the application is ready and given to the Test engineer, he/she will start checking every component of the module or module of the application independently or one by one, and this process is known as Unit testing Reduces Defects in the Newly developed features or reduces bugs when changing the existing functionality.

- Reduces Cost of Testing as defects are captured in very early phase.
- Improves design and allows better refactoring of code.
- Unit Tests, when integrated with build gives the quality of the build as well.

Unit Testing Lifecycle:



Advantages:-

- Early Detection of Defects
- Facilitates Refactoring
- Improved Code Quality
- Regression Testing
- Documentation and Design Clarification
- Faster Development Cycles
- Enables Continuous Integration
- Confidence in Code Changes

Disadvantages:-

- Time-Consuming
- Code Overhead
- Limited Testing Scope
- False Sense of Security
- Dependency on Test Quality
- Maintenance Overhead
- Difficulty in Testing External Dependencies
- Not Always Cost-Effective

6.2.1 Integration Testing:

Integration Testing is a crucial phase in the software development lifecycle where individual software modules are combined and tested as a group. In a typical software project, various modules are developed by different programmers, and integration testing aims to identify any defects or issues that arise from the interaction between these modules.

This testing level focuses on verifying the communication and data exchange between the integrated modules. Integration Testing is often referred to as 'I & T' (Integration and Testing),

'String Testing', or 'Thread Testing'. The process involves taking all the previously tested individual modules, integrating them together, and conducting further testing to ensure that they function correctly as per the user requirements. This testing phase checks parameters passing between the units and ensures the overall integrity and functionality of the software when all modules are integrated.

Strategies for Integration Testing:

- Big-Bang Integration
- Top Down Integration
- Bottom Up Integration
- Hybrid Integration

Integration testing for the child monitoring system using Android would involve ensuring that all components of the application work together seamlessly to achieve the desired functionality. Here's how integration testing could be approached:

Top-Down Integration Testing:

- Start with testing the highest-level modules of the application, such as the user interface and main functionalities visible to the parent.
- Ensure that the parent-facing features, such as viewing the child's location, messages, contacts, and call logs, are working correctly and display the expected information.
- Gradually integrate lower-level modules, such as database access and communication with the child's device, to verify that they interact correctly with the higher-level modules.

Bottom-Up Integration Testing:

- Begin with testing the lowest-level modules, such as the database access and network communication modules.
- Ensure that these modules function correctly and can retrieve data from the database or communicate with external devices as expected.
- Gradually integrate higher-level modules, such as the user interface components and parent-facing features, to verify that they interact correctly with the lower-level modules.

Hybrid Approach:

- Combine elements of both top-down and bottom-up integration testing to ensure comprehensive coverage of the application.
- Start by testing critical parent-facing features using the top-down approach to verify user experience and functionality.
- Simultaneously, conduct bottom-up testing to verify the functionality of individual components and their interactions.
- Gradually integrate modules from both ends, focusing on critical paths and ensuring smooth interactions between modules.

Big Bang Approach:

- Once individual components have been thoroughly tested using the other techniques, perform a final integration test using the Big Bang approach.
- Integrate all modules of the application simultaneously and verify that the application functions correctly as a whole.
- This approach allows for comprehensive system testing and validation before the final release of the application.

Functional Testing:

Functional Testing is a type of software testing where the system is evaluated based on its functional requirements and specifications. It verifies that the functions or features of the software perform as expected by feeding input into them and examining the output. The main objective of functional testing is to ensure that the application meets the specified requirements accurately. Identify functions that the software is expected to perform as expected by feeding input into them and examining the output. The main objective of functional testing is to ensure that the application meets the specified requirements accurately.

Unlike other types of testing, functional testing focuses solely on the outcomes of processing rather than how the processing occurs. It simulates real-world usage scenarios without making assumptions about the internal structure of the system. Functional testing employs the Black Box Testing technique, where the tester does not have access to the internal logic of the system being tested. Functional testing is typically conducted during the phases of System Testing and Acceptance Testing, ensuring that the software functions correctly before it is deployed to users.

Functional testing is more effective when the test conditions are created directly from user/business requirements. When test conditions are created from the system documentation (system requirements/ design documents), the defects in that documentation will not be

detected through testing and this may be the cause of end-users' wrath when they finally use the software.

6.3 Test Plan

A test plan is a document detailing the objectives, resources, and processes for a specific test for a software or hardware product. The plan typically contains a detailed understanding of the eventual workflow. A Test Plan is a detailed document that describes the test strategy, objectives, schedule, estimation and deliverables and resources required for testing. Test Plan helps us determine the effort needed to validate the quality of the application under test. The test plan serves as a blueprint to conduct software testing activities as a defined process which is minutely monitored and controlled by the test manager. A test plan documents the strategy that will be used to verify and ensure that a product or system meets its design specifications and other requirements. A test plan is usually prepared by or with significant input from test engineers. Depending on the product and the responsibility of the organization to which the test plan applies, a test plan may include a strategy for one or more of the following:

Our test plan for the Child Monitoring System Using Android project is meticulously crafted to ensure the quality and reliability of the application. We will prioritize testing core functionalities, conducting multiple testing levels including unit, integration, system, and acceptance testing. Various testing types such as functional, performance, security, usability, compatibility, and regression testing will be employed to ensure comprehensive coverage.

We will develop specific test cases for each feature, establishing clear schedules and milestones to track progress effectively. Our primary goal is to deliver a reliable and user-friendly Child Monitoring System application, with continuous improvements and enhancements incorporated during the testing cycles.

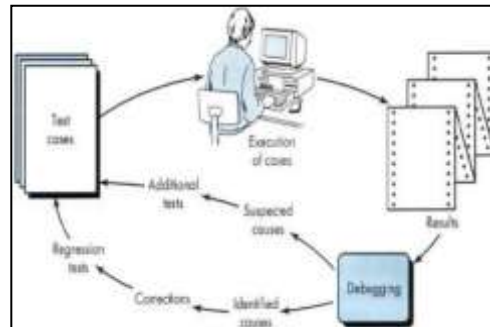
Regression test – to be performed on an existing operational product, to verify that existing functionality was not negatively affected when other aspects of the environment were changed (e.g., upgrading the platform on which an existing application runs). A complex system may have a high-level test plan to address the overall requirements and supporting test plans to address the design details of subsystems and components.

Test plan document formats can be as varied as the products and organisations to which they apply. There are three major elements that should be described in the test plan; Test Coverage, Test Methods, and Test Responsibilities. These are also used in a format test strategy.

6.4 Debugging Approach

It is an error that results in the removal of error. Debugging always occurs as a consequence of testing. Debugging will always have one of two outcomes. The cause will be found and corrected. The cause will not be found. In the latter case, the person performing debugging may

suspect a cause, design one or more test cases to help validate that suspicion, and work toward error correction in an iterative fashion.



Debugging Strategies: In general, 3 debugging strategies have been proposed:

1. Brute force
2. Back Tracking
3. Cause Elimination

6.4.1 Brute Force Approach

This method is most common and least efficient for isolating the cause of a software error. We apply this method when all else fail. In this method, a printout of all registers and relevant memory locations is obtained and studied. All dumps should be well documented and retained for possible use on subsequent problems. In our project we used this approach to test the login page.

6.4.2 Backtracking Approach

It is a quite popular approach of debugging which is used effectively in case of small applications. The process starts from the site where a particular symptom gets detected, from there on backward tracing is done across the entire source code till we are able to lay our hands on the site being the cause. Unfortunately, as the number of source lines increases, the number of potential backward paths may become unmanageably large.

6.4.3 Cause Elimination

The third approach to debugging, cause elimination, is manifested by induction or deduction and introduces the concept of binary partitioning. This approach is also called induction and deduction. Data related to the error occurrence are organized to isolate potential causes. A “cause hypothesis” is devised and the data are used to prove or disprove the hypothesis. Alternatively, a list of all possible causes is developed and tests are conducted to eliminate each. If initial tests indicate that a particular cause hypothesis shows promise, the data are refined in an attempt to isolate the bug. In our project we didn’t mostly follow this process as it can eliminate the required things for others.

6.5 Test Cases

LEFT TO DO

6.5.1 Features to be tested

1. Login Form
2. Access Location Of Child
3. Access Call Logs Of Child
4. Access Messages Of Child
5. Access Contacts Of Child

Conclusion

In conclusion, the testing and debugging approach for the project plays a critical role in guaranteeing the reliability, accuracy, and functionality of the system. Through systematic testing processes and diligent debugging efforts, potential software defects and inconsistencies are identified, isolated, and resolved to maintain the system's performance standards.

Additionally, clear roles and responsibilities, along with proactive risk management strategies, contribute to effective testing execution and oversight. By prioritizing thorough testing and debugging practices, the project aims to deliver a robust and dependable solution that meets the needs of its users and stakeholders.

Chapter 6: Snapshots

7.1. Snapshots

Chapter 8: Snapshots

Content:

8.1 Introduction

8.2 Limitation

8.3 Future Enhancement

8.4 Reference and Bibliography